

RV UNIVERSITY

RV Vidyanikethan Post, 8th Mile, Mysuru Road, Bengaluru – 560 059, Karnataka, India

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



A Project Report On

MemoryGraph AI: A Dynamic, Context-Aware Hybrid Knowledge Graph Framework for Documents with Comparative Vector Database Analysis

Submitted in partial fulfillment for the award of degree of

B.Tech(Honors)

in

School of Computer Science and Engineering

CS4905 - Major Project

Submitted By

Amrutha M	1RVU22CSE020
Bhargava Srinivasan	1RVU22CSE036
Renu Bojja	1RVU22CSE129

Under the Guidance of

Dr. Shouvik Chakraborty

Assistant Professor

School of Computer Science and Engineering

RV University, Bengaluru-560059

Jan – May 2026

RV UNIVERSITY

RV Vidyanikethan Post, 8th Mile, Mysuru Road, Bengaluru – 560 059, Karnataka, India

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to be certified that the project work titled **MemoryGraph AI: A Dynamic, Context-Aware Hybrid Knowledge Graph Framework for Documents with Comparative Vector Database Analysis** is carried out by **Amrutha M (1RVU22CSE020), Bhargava Srinivasan (1RVU22CSE036), Renu Bojja (1RVU22CSE129)**, a bonafide students of RV University, Bengaluru, in partial fulfilment of **Bachelor of Technology (Hons) in School of Computer Science and Engineering** of the RV University, Bengaluru during the year 2025-2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the major project report deposited in the departmental library. The Project report has been approved as it satisfies the academic requirements in respect of major project work prescribed by the institution for the said degree.

Dr. Shouvik Chakraborty

Guide

Dr K C Narendra

**Program Director, B. Tech
(Hons)**

Prof. G Shobha

Dean, SoCSE

External Viva:

Name of Examiners

Signature with Date

1.

2.

DECLARATION

We, **Amrutha M (1RVU22CSE020)**, **Bhargava Srinivasan (1RVU22CSE036)**, **Renu Bojja (1RVU22CSE129)**, students of Eighth semester B.Tech(Hons), School of Computer Science and Engineering, RV University, Bengaluru, hereby declare that the major project titled **MemoryGraph AI: A Dynamic, Context-Aware Hybrid Knowledge Graph Framework for Documents with Comparative Vector Database Analysis** has been carried out by us and submitted in partial fulfilment of **Bachelor of Technology (Hons)** in **School of Computer Science and Engineering** during the year 2025-26.

Further we declare that the content of the report has not been submitted previously by anybody or to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RV University will be the property of RV University, Bengaluru and we will be one of the authors of the same.

Place: Bengaluru

Date: 23-04-2026

Name	Signature
Amrutha M (1RVU22CSE020)	
Bhargava Srinivasan (1RVU22CSE036)	
Renu Bojja (1RVU22CSE129)	

ACKNOWLEDGEMENT

We express our sincere and heartfelt gratitude to **Dr. Shouvik Chakraborty**, project guide for the constant guidance, valuable suggestions, technical expertise, and continuous encouragement provided throughout the course of our project work.

We would also like to extend our sincere thanks to **Prof. Sarasvathi V, Dr. Basavaraj Patil, and Dr. Sheela S**, Project Coordinators for their timely support, monitoring, and valuable inputs during various phases of the project.

We are deeply grateful to the **Dr. K C Narendra**, Program Director, B. Tech (Hons) for the academic encouragement and support extended to us in successfully carrying out this project work. We take this opportunity to express our profound gratitude to **Prof. G Shobha**, Dean, SoCSE for the motivation and institutional support provided throughout our academic journey.

We also extend our sincere thanks to **Prof. Sahana D. Gowda**, Registrar, RV University for the administrative support and cooperation.

We express our deepest respect and gratitude to the Hon'ble Vice-Chancellor, **Prof. Ram Kumar Kakani**, Vice-Chancellor, RV University for the visionary leadership and for fostering a research-oriented and innovative academic environment.

Further, we extend our heartfelt thanks to the Management of RV University for providing excellent infrastructure, state-of-the-art facilities, and a conducive learning environment that enabled us to complete this project successfully.

We are also grateful to our family and friends who provided us with every requirement throughout the course. We would like to thank one and all who directly or indirectly helped us in the Project work.

ABSTRACT

The rapid growth of unstructured digital documents has created significant challenges for information retrieval and knowledge extraction systems. Traditional Retrieval-Augmented Generation (RAG) frameworks rely primarily on vector-based semantic search to retrieve relevant document chunks, but they often fail to capture the relationships between entities distributed across multiple documents. This limitation leads to fragmented context and reduced effectiveness in complex question answering tasks that require multi-hop reasoning. To address these challenges, this project proposes MemoryGraph AI, a framework that integrates knowledge graph construction with retrieval-augmented generation to enable structured knowledge representation and contextual reasoning across documents.

The proposed system extracts entities and relationships from uploaded documents and organizes them into a persistent knowledge graph stored in a Neo4j graph database. Unlike conventional systems that process documents independently, MemoryGraph AI supports multi-document knowledge graph construction, enabling connections between entities originating from different sources when meaningful relationships exist. The framework further incorporates embedding-based semantic retrieval using sentence transformers, allowing efficient identification of relevant information. By combining graph traversal with vector similarity search, the system enables cross-document contextual retrieval, where answers are generated using interconnected knowledge from multiple documents.

The proposed architecture adopts a hybrid retrieval approach that leverages the complementary strengths of knowledge graphs and vector embeddings. The knowledge graph provides structured contextual relationships between entities, while embeddings enable fast semantic similarity search across large document collections. Additionally, the system maintains a persistent graph-based knowledge memory, allowing knowledge extracted from documents to be reused across multiple queries without repeated processing. This approach improves contextual coherence, supports multi-hop reasoning, and enhances the overall efficiency of document-based question answering systems.

TABLE OF CONTENTS

Acknowledgement	i
Abstract	ii
Table of Contents	iii
List of Figures	v
List of Tables	vi
List of Abbreviations	vii
Chapter 1 INTRODUCTION	1
1.1 Background	1
1.2 Real world relevance	2
1.3 Current challenges and limitations	3
1.4 Organization of the report	3
Chapter 2 LITERATURE SURVEY	5
2.1 Review of existing research work	5
2.2 Comparative analysis	7
2.3 Research gap	12
Chapter 3 PROBLEM STATEMENT	15
3.1 Motivation	15
3.2 Problem Definition	16
3.3 Objectives	17
3.4 Scope of the project	17
Chapter 4 SOFTWARE REQUIREMENTS SPECIFICATION	19
4.1 Functional requirements	19

4.2	Non-functional Requirements	20
4.3	Operating environment	21
Chapter 5	PROPOSED METHODOLOGY	24
5.1	System Architecture	25
5.2	Modules	26
5.3	Novel features of proposed system	30
5.4	Algorithms	33
Chapter 6	SYSTEM DESIGN	35
6.1	Class diagram	35
Chapter 7	IMPLEMENTATION	38
7.1	Modules implemented	38
7.2	Execution flow	46
Chapter 8	RESULTS AND DISCUSSION	49
8.1	Output Screenshots	49
8.2	Results with interpretations	52
Chapter 9	CONCLUSION AND FUTURE SCOPE	58
9.1	Conclusion	58
9.2	Future Scope	59
	BIBLIOGRAPHY	60
	APPENDIX A: PUBLICATION PAPER	62
	APPENDIX B: PLAGIARISM REPORT	68

LIST OF FIGURES

		Page No.
Figure 5.1	System Architecture of MemoryGraph AI	25
Figure 6.1	Class Diagram of MemoryGraph AI	37
Figure 7.1	Execution Flow Diagram of MemoryGraph AI	46
Figure 8.1	MemoryGraph AI User Interface	49
Figure 8.2	Knowledge Graph Insights and Entity Analysis	50
Figure 8.3	Interactive Knowledge Graph Visualization	50
Figure 8.4	Graph Evidence Supporting Generated Answers	51
Figure 8.5	Performance Metrics	51
Figure 8.6	Neo4j Graph Representation	56

LIST OF TABLES

		Page No.
Table 2.1	Comparitive Analysis	8
Table 7.1	Modules Implemented	38
Table 8.1	Performnace Metrics	53
Table 8.2	Comparison Analysis	54

LIST OF ABBREVIATIONS

Abbreviation	Definition
RAG	Retrieval Augmented Generation
LLM	Large Language Model
GNN	Graph Neural Network
KG	Knowledge Graph
NLP	Natural Language Processing

Chapter 1

INTRODUCTION

With the exponential growth in the volume of digital documents, there is a need for effective knowledge extraction and retrieval techniques. Conventional approaches such as keyword-based searching and vector similarities have limitations since they cannot recognize relations between entities across different documents. This chapter discusses the background and practical significance of multi-document knowledge extraction, as well as its associated challenges and possible solutions, particularly through knowledge graphs, embeddings, and language models.

1.1 Background

In recent years, with the fast expansion of digital information and scientific publications, there is an increasing challenge in knowledge extraction from massive collections of documents. Researchers and academic institutions routinely encounter numerous documents pertaining to correlated concepts, datasets and research methods. Existing information retrieval methods based on keyword search or basic vector similarity cannot capture the deeper semantic relationship between concepts across documents.

The recent advances in artificial intelligence and natural language processing, as well as the use of knowledge graphs to represent the relationship between entities in textual data, have paved the way for the development of intelligent systems capable of capturing and reasoning over entities in textual data for knowledge extraction. Knowledge Graph is a graph representation of data in which nodes represent entities and edges represent relationships between entities. Knowledge graphs can facilitate a machine to capture the structured semantic relationship between concepts.

Recent developments in graph databases such as Neo4j and vector embeddings have enabled hybrid retrieval systems that combine semantic similarity with structured relational

reasoning. These technologies allow systems to move beyond simple document retrieval and provide context-aware insights from large datasets.

MemoryGraph AI is a multi-document question answering system that uses knowledge graph, vector embedding and large language model to answer user queries by extracting relationships and storing them in graph and generating context-aware answer from large knowledge base.

1.2 Real world relevance

Contemporary research environments generate a considerable amount of scientific literature, technical reports and digital documents. Typically, to understand the relationship between research methods, datasets and experimental results, researchers spend a considerable amount of time perusing over several papers. While traditional search engines retrieve documents based on the relevance of the document, they do not provide structured insights into how research concepts are related to each other.

Many domains rely on the ability to analyze large collections of textual documents. For instance, in the biomedical domain, researchers must examine thousands of research papers to discover the relationships between various diseases, treatments and experimental results. Similarly, in the domain of cybersecurity, analysts must read through multiple reports to identify patterns of attacks and/or vulnerabilities.

A system that can automatically construct a knowledge graph from documents and facilitate intelligent querying can significantly enhance research productivity. The proposed system can help users potentially identify hidden relationships, summarize knowledge across documents, and answer complex research queries efficiently.

1.3 Current challenges and limitations

Introduction: Despite the progress in information retrieval and machine learning, current document analysis systems have several limitations. Most current systems use pure keyword-based search or vector similarity search to retrieve documents, which do not explain the relationships of retrieved concepts.

While vector databases allow for semantic search, they do not model the relationships between entities explicitly. They thus cannot reason about more complex problems that involve multi-step chains of reasoning. Additionally, many systems do not preserve contextual information across multiple documents, which limits their application in research analysis.

Scalability and interpretability pose additional challenges. Many deep learning based systems are black boxes, making it difficult to understand how conclusions are reached based on the input data. Additionally, the integration of structured reasoning with semantic retrieval is an open research problem.

These observations suggest that hybrid systems that combine vector-based semantic search with graph-based relational reasoning are essential to allow for more effective multi-document knowledge discovery.

1.4 Organization of the report

This report is organized into several chapters that describe the development and evaluation of the proposed system.

Chapter 1 introduces the background, relevance, and challenges associated with multi-document knowledge extraction systems.

Chapter 2 presents a detailed literature survey analyzing existing research work and identifying gaps in current methodologies.

Chapter 3 defines the problem statement, motivation, objectives, and scope of the proposed work.

Chapter 4 describes the software requirements specification including functional and non-functional requirements and the operating environment.

Chapter 5 explains the proposed methodology, system architecture, modules, and algorithms used to build the system.

Chapter 6 presents the system design using UML diagrams such as use case diagrams, class diagrams, and sequence diagrams.

Chapter 2

LITERATURE SURVEY

This chapter provides an extensive literature review of research conducted so far in relation to retrieval-augmented generation techniques, knowledge graph building, and document-based QA systems. It discusses critical findings, examines various techniques, and identifies their capabilities and constraints. Based on this analysis, the chapter identifies important research gaps that motivate the development of the proposed MemoryGraph AI framework.

2.1 Review of existing research work

Recent studies have attempted to leverage the use of knowledge graphs, large language models, and retrieval-augmented generation in improving the process of information retrieval and answering questions from a large set of documents. Conventional systems use vector similarity search as the primary method of retrieval systems. These systems may retrieve relevant documents but cannot handle relationships between entities. To overcome these problems, recent studies have attempted to leverage the use of knowledge graphs in improving retrieval systems.

The concept of Retrieval-Augmented Generation (RAG), proposed in a research paper by Lewis et al., can be considered one of the first attempts at combining document retrieval with language generation systems. In RAG systems, a retriever component of the model is used to retrieve documents using dense vector embeddings. A generator component of the model generates an answer based on the context retrieved. Although RAG systems have shown promising results in improving the performance of language models in knowledge-intensive tasks, these systems do not leverage relationships between entities in the context of a document. RAG systems may face problems in answering multi-hop queries since the documents retrieved may not have a relationship with each other.

To overcome the problems of RAG systems in answering multi-hop queries, recent studies have attempted to leverage the use of knowledge graphs in improving the process of retrieval

systems. For example, the KG²RAG framework has proposed the use of knowledge graphs in expanding the context of a text chunk retrieved from a retrieval system. Similarly, the GeAR framework has proposed a graph-enhanced agent in improving the process of multi-hop retrieval using semantic triples between related documents. These studies have shown the effectiveness of using knowledge graphs in improving the process of retrieval systems. However, these systems have used static knowledge graphs in improving retrieval systems.

Some studies have also explored the problem of transforming unstructured documents into knowledge graphs. For example, the Docs2KG framework uses a variety of unstructured documents such as PDF documents, emails, and web pages. It uses large language models and layout analysis techniques to transform all the documents into a single knowledge graph. The AutoKG framework uses a similar approach to the previous one. It uses automatic knowledge graph construction techniques such as OpenIE and contextual entity linking using BERT embeddings. Although the above frameworks are able to extract knowledge from unstructured documents, they are mostly limited to single-document processing. They are not able to fully handle the problem of knowledge integration across multiple documents. In addition, they are not able to dynamically update the knowledge graph when multiple queries are posed on the knowledge graph.

There are other studies that have explored the problem of hybrid search systems. For example, the HybridRAG framework uses a combination of knowledge graphs and semantic vectors. It uses the combination of knowledge graphs and semantic vectors to enhance the performance of the information extraction system. The MegaRAG framework uses a combination of knowledge graphs and semantic vectors. It uses a combination of knowledge graphs and semantic vectors to construct a multimodal knowledge graph. It uses the knowledge graph and the pages of the documents to perform a two-layer retrieval.

Other survey papers on knowledge graph construction and graph-based reasoning also identify some open issues. Knowledge Graph Construction: Extraction, Learning, and Evaluation and A Comprehensive Survey on Automatic Knowledge Graph Construction are

two survey papers on knowledge graph construction. These survey papers emphasize the importance of scalable knowledge graph construction, integration of knowledge across documents, and evaluation. These survey papers also identify some open issues, such as the inefficiency in the integration of knowledge graphs and vector retrieval systems.

From the survey on the available literature, it is clear that there is significant progress in the integration of knowledge graphs, retrieval systems, and large language models for applications requiring a significant amount of knowledge. However, there are some significant gaps in the available research, such as the absence of persistent memory for knowledge, and the inefficiency in the integration of graph reasoning and vector retrieval.

2.2 Comparative analysis

Existing approaches for document-based question answering systems can be broadly categorized into keyword-based retrieval systems, vector similarity search systems, and graph-based reasoning systems. Systems that use keywords are efficient computationally, but they lack in semantic understanding. Vector based systems capture semantic similarity, but they often miss clear connections between entities.

Graph-based systems provide strong relational reasoning capabilities but sometimes struggle with discovering relevant nodes when queries use different terminology. Hybrid systems that combine vector embeddings with graph traversal have shown promise in addressing these limitations.

A comparative analysis of existing systems highlights trade-offs between scalability, semantic understanding, and reasoning capabilities. While vector databases excel in semantic search performance, knowledge graphs provide structured reasoning that is essential for complex queries involving relationships between entities.

Table 2.1 Comparative analysis

Sl. No.	Title & Author	Methodology / Approach	Key Contribution	Research Gap Identified	Suggestive Enhancements
1	Graph-Anchored Knowledge Indexing for RAG – Zhenghao Liu et al.	GraphAnchor builds a dynamic knowledge graph during iterative retrieval.	Graph-based indexing improves multi-hop retrieval reasoning.	Graph quality depends heavily on LLM extraction accuracy.	Combine graph indexing with vector database retrieval.
2	Graph Neural Networks: A Review of Methods and Applications – Chi-Hsiang Hsiao et al.	Survey of GNN architectures and graph learning techniques.	Provides overview of graph representation learning methods.	Limited application to document-based knowledge graphs.	Apply GNN-based reasoning for document knowledge graphs.
3	MegaRAG: Multimodal Knowledge Graph-Based RAG – Jie Zhou et al.	Constructs multimodal knowledge graphs from text, images, and tables.	Enables hybrid retrieval using graph and document pages.	Knowledge graph not dynamically updated.	Introduce evolving graph memory for document collections.
4	KG ² RAG: Knowledge Graph-Guided RAG – Xiangrong Zhu et al.	Uses KG traversal to expand retrieved document chunks.	Improves context organization and multi-hop retrieval.	Static knowledge graph built during preprocessing.	Enable dynamic graph updates and hybrid retrieval.

5	GeAR: Graph-Enhanced Agent for RAG – Zhili Shen et al.	Agent-based retrieval using graph triples and iterative reasoning.	Introduces graph-based multi-hop retrieval and memory summaries.	Static triple-based graph limits adaptability.	Implement dynamic graph construction and entity refinement.
6	Knowledge Graph Construction: Extraction, Learning, and Evaluation – Seungmin Choi et al.	Survey of KG pipelines including extraction, learning, and evaluation.	Comprehensive KG evaluation framework for quality assessment.	Knowledge graphs mostly static and lack hybrid retrieval.	Integrate vector search with dynamic graph structures.
7	Docs2KG: Unified KG Construction from Heterogeneous Documents – Qiang Sun et al.	Converts PDFs, web pages, and structured data into unified knowledge graphs.	LLM-assisted multimodal extraction from documents.	Limited evaluation with vector retrieval systems.	Combine KG construction with hybrid graph-vector retrieval.
8	AutoKG: Constructing Virtual Knowledge Graphs from Documents – Seunghak Yu et al.	Automatic KG generation using OpenIE and contextual embeddings.	Enables multi-hop QA using automatically generated graphs.	Limited cross-document knowledge integration.	Introduce cross-document graph linking and dynamic updates.

9	RAKG: Retrieval-Augmented Knowledge Graph Construction – Hairong Zhang et al.	Integrates retrieval with KG construction using NER-based pre-entities.	Improves entity extraction accuracy using contextual retrieval.	No persistent knowledge graph across documents.	Develop unified multi-document graph memory.
10	Survey on Automatic Knowledge Graph Construction – Lingfeng Zhong et al.	Survey of KG extraction, graph learning, and evaluation techniques.	Introduces taxonomy for knowledge graph construction methods.	Limited exploration of cross-document graph integration.	Develop scalable cross-document knowledge graphs.
11	KG-BERT: BERT for Knowledge Graph Completion – Liang Yao et al.	Uses BERT to model KG triples as textual sequences.	Context-aware representation of entities and relations.	Does not address KG construction from documents.	Integrate KG completion with document extraction pipelines.
12	HybridRAG: Integrating Knowledge Graphs and Vector Retrieval – Bhaskarjit Sarmah et al.	Combines graph-based reasoning with vector retrieval.	Demonstrates hybrid retrieval architecture for QA systems.	Limited cross-document graph integration.	Extend hybrid retrieval for multi-document reasoning.

13	E2GraphRAG: Efficient Graph-based RAG – Yibo Zhao et al.	Uses tree- graph hybrid structure with adaptive retrieval.	Improves retrieval efficiency with graph-aware strategies.	No persistent knowledge graph memory across queries.	Introduce long- term graph memory structures.
14	Retrieval- Augmented Generation for Knowledge- Intensive NLP – Patrick Lewis et al.	Combines dense retrieval with transformer- based generation.	Introduces the RAG architecture for knowledge retrieval.	Lacks structured knowledge representation.	Integrate knowledge graphs with RAG pipelines.
15	LLMs Meet Knowledge Graphs for Question Answering – Chuangtao Ma et al.	Studies integration of LLMs with knowledge graphs.	Provides taxonomy for KG-augmented LLM reasoning systems.	Scalability challenges and limited hybrid retrieval integration.	Develop scalable graph- vector hybrid reasoning systems.
16	GNN-RAG: Graph Neural Retrieval for LLM Reasoning – Costas Mavromatis et al.	Combines graph neural networks with RAG retrieval.	Enables graph- based multi-hop reasoning for QA tasks.	High computational complexity and graph dependency.	Improve scalability using hybrid retrieval strategies.

17	DO-RAG: Domain-Specific QA using KG-Enhanced RAG – David Osei Opoku et al.	Combines knowledge graphs and vector retrieval for QA.	Improves domain-specific QA accuracy with hybrid retrieval.	Domain-specific evaluation limits generalization.	Apply framework to multi-domain document datasets.
----	--	---	---	--	--

2.3 Research gap

The literature indicates significant advancements in the understanding of documents and answering of questions through the use of Retrieval-Augmented Generation (RAG), knowledge graphs, and large language models. There is a lot of recent work combining the use of knowledge graphs and retrieval systems in improving contextual reasoning and multi-hop question answering. Systems such as KG²RAG, GeAR, HybridRAG, and MegaRAG show the promise of combining the use of knowledge graphs with retrieval systems in improving the retrieval process through the use of entities and relationships extracted from texts. On the other hand, there are a lot of studies aimed at developing knowledge graphs from unstructured documents using OpenIE extraction techniques, transformer models, and graph neural networks.

However, despite these advancements in the use of knowledge graphs in retrieval systems, there are a number of significant limitations in the use of these systems. Firstly, one of the major limitations of these systems is the lack of a permanent memory of the knowledge graph. Most of these frameworks develop a knowledge graph during the preprocessing phase and discard it after the retrieval process or develop a new one during each session. Therefore, these systems cannot have a permanent memory of the knowledge graph that accumulates new insights from new documents.

Another significant limitation of these systems is the lack of proper integration of the knowledge graph across multiple documents. Most of these systems develop a knowledge graph at the document level and treat each document separately without integrating entities across documents. Therefore, answering a question through cross-document reasoning—where the answer requires combining entities from multiple documents—becomes a challenge because of the lack of a unified graph.

The drawback of the current state of retrieval systems is that graph reasoning and vector-based semantic search are not well combined. There are some systems that use knowledge graphs to manage the retrieved contexts. However, the drawback of the current state of retrieval systems is that graph reasoning and vector-based semantic search are not well combined. There are some systems that use knowledge graphs to manage the retrieved contexts. However, the drawback of the current state of retrieval systems is that graph reasoning and vector-based semantic search are not well combined. There are some systems that use knowledge graphs to manage the retrieved contexts. However, the drawback of the current state of retrieval systems is that graph reasoning and vector-based semantic search are not well combined. There are some systems that use knowledge graphs to manage the retrieved contexts.

Furthermore, most of the proposed solutions in the literature have been applied to specific domains. For example, the proposed solutions have been applied to financial documents or specific knowledge bases. However, the drawback of the proposed solutions is that they have been applied to specific domains. Therefore, the proposed solutions have the drawback that they might not be effective in handling various documents. Additionally, most of the proposed solutions use static graph structures. Therefore, the drawback of the proposed solutions is that they might not be effective in handling various documents. Therefore, the proposed solutions have the drawback that they might not be effective in handling various documents. Additionally, most of the proposed solutions use static graph structures. Therefore, the drawback of the proposed solutions is that they might not be effective in

handling various documents. Therefore, the proposed solutions have the drawback that they might not be effective in handling various documents.

Thus, there exists a definite requirement for a system that can overcome these limitations by facilitating persistent knowledge memory, knowledge integration across documents, and hybrid retrieval strategies. The proposed MemoryGraph AI system seeks to fill these research gaps by building a persistent and dynamically evolving knowledge graph from a set of documents and integrating graph-based reasoning with vector-based semantic retrieval. This would facilitate context-aware multi-document reasoning with efficient retrieval performance, thus enhancing the efficacy of question answering systems based on documents.

Chapter 3

PROBLEM STATEMENT

This chapter describes the main challenge that will be addressed using the proposed system. It starts by discussing the challenges associated with conventional techniques for document retrieval and knowledge acquisition. It then formulates the problem statement, and specifies the objectives of the work. Additionally, the scope of the project is discussed to clarify the boundaries and expected outcomes of the MemoryGraph AI system.

3.1 Motivation

The result of the explosion in digital information has led us to a point where we are constantly producing huge amounts of documents, research papers, technical reports, and web pages. It has become a huge nuisance to extract useful information from all such sources and make it readily available. The traditional information retrieval task has been based on keyword matching or vector similarity to extract relevant information from text. These approaches can identify semantically similar documents, but they do not understand the relationships between entities and concepts that are implicit in the text.

Recently, Retrieval Augmented Generation (RAG) and large language models have helped move information retrieval forward. These models are more effective at retrieving information from massive quantities of text. Many RAG models, however, still tend to treat individual documents as separate units, retrieving information from these documents in disjointed ways, without understanding how individual pieces of information relate to one another.

Knowledge graphs are useful in providing a structured view of how different entities are related to each other. This is useful in helping us better organize our information. Using knowledge graphs along with language models and retrieval systems can help us achieve more accurate contextual reasoning and better responses. However, existing techniques are

static and do not allow us to store information from past sessions or effectively combine graph-based reasoning with vector-based semantic search techniques.

This has led to the development of MemoryGraph AI - a hybrid retrieval system that combines knowledge graph construction techniques with vector-based embeddings and large language models.

3.2 Problem definition

Although there have been major breakthroughs in document retrieval and language models, there are still some challenges in building effective document understanding systems. The conventional document retrieval systems are primarily based on semantic similarity search using vector embeddings, which are capable of retrieving relevant documents but are not able to extract the relationships between entities. Therefore, these systems are not able to answer questions that involve reasoning about multiple pieces of information.

Another major challenge is the lack of persistent knowledge representation. Many systems process documents temporarily during retrieval and discard the extracted knowledge after the session ends. This prevents the system from accumulating knowledge over time and requires repeated processing of the same documents.

In addition, the existing systems tend to build the knowledge graph for individual documents separately, without considering the entities and relationships from multiple documents. This drawback makes it difficult to reason across documents and find relationships between concepts from different sources.

Thus, the problem being solved in this project is to design and implement a system capable of automatically extracting structured knowledge from documents, building a persistent knowledge graph, and integrating graph-based reasoning with semantic retrieval approaches to enable efficient multi-document question answering.

3.3 Objectives

The main aim of this project is to design and develop a hybrid knowledge retrieval system that can extract, organize, and retrieve information from various documents using knowledge graphs and large language models.

The specific objectives of the proposed system are as follows:

- Design a knowledge extraction system that can automatically extract entities and relationships from the content of documents using large language models.
- Design a structured knowledge graph system that can organize the extracted entities and their relationships in a graph database.
- Design a vector embedding system for graph entities to enable efficient semantic similarity search.
- Design a hybrid retrieval system that can combine knowledge graph search and vector semantic search.
- Design a cross-document reasoning system that can integrate entities and relationships extracted from various documents.
- Design a persistent knowledge graph memory system that can store information from multiple sessions and enable incremental knowledge updates.
- Design an interactive system that can enable users to upload documents, query the knowledge graph, and visualize system outputs.

3.4 Scope of the project

The MemoryGraph AI project scope is centered on creating a system that is capable of extracting structured knowledge from documents and answering questions using a hybrid retrieval framework.

The system begins by processing documents. The text is extracted from the document, and a large language model is used to parse the text. The knowledge is stored in a graph database, where the entities are nodes that are linked by relationships. To improve the search process, vector embeddings are used for the entities of the graph database.

When a user wants to know something, a hybrid technique is used that retrieves information from vector search and a graph database. The information is then used by a language model to generate accurate responses that are contextually aware. The project is built to work in multiple domains and does not require domain-specific data. This allows the system to handle different document collections and construct knowledge graphs from different sources of information.

The current project scope is limited to knowledge extraction and retrieval from documents. Advanced functionalities such as distributed graph processing, real-time knowledge updates, and conversational memory integration are beyond the scope of the current project and can be explored in future work.

Chapter 4

SOFTWARE REQUIREMENTS SPECIFICATION

The chapter describes the software requirements for the proposed system, taking into account not only the functional requirements but also the non-functional ones. The characteristics of the system are described along with the requirements for performance, usability, and reliability. Also, the environment where the system operates will be described in order to understand what hardware and software must be used for implementing the MemoryGraph AI system.

4.1 Functional requirements

Functional requirements are the operations and services that the system must provide in order to fulfill its intended objectives. The MemoryGraph AI system is intended to process documents, build a knowledge graph, and enable intelligent question answering capabilities using hybrid retrieval approaches.

The major functional requirements of the system are as follows:

- The system should extract textual content from uploaded documents and perform necessary preprocessing operations.
- The system should automatically extract entities and relationships from document content using a large language model.
- The system should construct a knowledge graph from the extracted entities and relationships.
- The system should store the constructed knowledge graph in a Neo4j graph database for efficient storage and retrieval.
- The system should generate vector embeddings for entities in the knowledge graph to enable semantic similarity search.
- The system should create a vector index in the graph database to support fast embedding-based retrieval.

- The system should allow users to submit natural language queries through the user interface.
- The system should perform hybrid retrieval, combining vector similarity search with knowledge graph traversal.
- The system should generate responses to user queries using a large language model based on the retrieved graph context.
- The system should display the retrieved graph evidence used to generate the answer.
- The system should provide performance metrics such as ingestion time, extraction time, embedding time, and query latency.

4.2 Non-functional requirements

Non-functional requirements specify the quality attributes and operational characteristics that the system must satisfy.

Performance

The system should be capable of processing multiple documents efficiently and generating responses within an acceptable time frame. Query responses should ideally be generated within a few seconds depending on the size of the knowledge graph.

Scalability

The system should be able to process an ever-increasing number of documents and knowledge graph nodes without any degradation in performance. The application of graph databases and vector indexing enables scalable knowledge retrieval.

Reliability

Document uploads, knowledge extraction, and data storage in the graph database should all be reliably handled by the system without data loss.

Usability

The system should offer a simple web interface for users to upload documents, construct the knowledge graph, and enter queries without needing technical knowledge.

Maintainability

The modular design of the system enables individual system components like document processing, extraction, building, and querying to be changed or improved separately.

Security

API keys and database passwords should be stored in a secure manner using environment variables to protect against unauthorized access.

4.3 Operating environment

4.3.1 Hardware Requirements

Processor:	Intel Core i5 or higher
RAM:	Minimum 8 GB (16 GB recommended)
Storage:	Minimum 256 GB SSD
GPU:	Optional for running LLMs locally
Network:	Stable internet connection

4.3.2 Software Requirements

Operating System

The system can run on major operating systems that support Python and Neo4j.

- Windows 10 / Windows 11
- Linux (Ubuntu recommended)
- macOS

Programming Language

The entire system is developed using:

- **Python 3.9 or later**

Python is utilized for developing the document ingestion pipeline, knowledge graph building, embedding computation, and query answering components.

Libraries

The following Python libraries are required for the implementation of various components of the system:

Core AI / NLP Libraries

- **LangChain Core** – used for management of prompt and parsing of outputs.

- **LangChain Groq** – This library is used for the integration of the Groq LLM API for knowledge extraction and question-answering.
- **SentenceTransformers** – used for the generation of vector embeddings for graph entities.

Data Processing Libraries

- **PyPDF2** - This library is used to extract text from PDF files during data ingestion.
- **python-docx** - This library is used to extract text from DOCX files.
- **tqdm** - This library is used to display progress bars during data ingestion.
- **Pandas** - This library is used to display evaluation metrics in the dashboard.

Graph Processing Libraries

- **Neo4j Python Driver** - Enables interaction between Python and the Neo4j graph database.

Utility Libraries

- **Pydantic** - Used for defining structured data models for knowledge graph extraction.
- **python-dotenv** - Loads environment variables such as API keys and database credentials.

Frameworks

The project uses the following framework for building the user interface:

- **Streamlit** – Used to build an interactive web interface for uploading documents, querying the graph, and visualizing system metrics.

Database

The system uses a graph database to store extracted entities and relationships.

- **Neo4j Graph Database**

Neo4j stores entities as nodes and relationships as edges and also supports vector indexing for semantic similarity search.

Tools

The following development tools are used for implementation and deployment:

- **Visual Studio Code (VS Code)** – Code development and debugging
- **Git** – Version control system

- **Neo4j Browser / Neo4j Aura Console** – Used for managing and visualizing the graph database.

Cloud / API Services

The system integrates cloud-based services for language model inference:

- **Groq API** – Used to access the LLaMA-3 language model for entity and relationship extraction and question answering.

Model used:

- **LLaMA-3.1-8B-Instant**

Chapter 5

PROPOSED METHODOLOGY

This chapter presents the proposed methodology for the MemoryGraph AI system, which is designed to improve document understanding and question answering using a hybrid knowledge retrieval framework. The system integrates knowledge graph construction, vector embeddings, and large language models to support context-aware multi-document reasoning.

The proposed MemoryGraph AI framework includes a formal comparative evaluation between two retrieval architectures: pure Vector RAG, which encodes queries using the SentenceTransformer library and retrieves the top-K most semantically similar entities via cosine similarity over stored embeddings, and Hybrid Graph RAG, which extends vector retrieval by traversing the knowledge graph from each seed entity to collect one-hop neighbours, assembling structured relational triples as context for the LLM. Both systems operate over the same entity embedding space, ensuring that performance differences are attributable solely to the graph traversal step. The systems are evaluated across nine queries using retrieval ranking metrics (MRR, nDCG@K, Entity Coverage) and LLM-as-Judge metrics assessing answer Faithfulness, Completeness, and Context Structural Richness on a 1–5 scale. The Hybrid Graph RAG approach outperforms Vector RAG on 9 of 11 evaluated metrics, achieving a 104% improvement in Entity Coverage (0.954 vs 0.467), a +1.44 point gain in Structural Richness, and higher Faithfulness and Completeness scores, with zero Vector RAG wins. The hybrid approach is therefore adopted as the final system, as it demonstrably retrieves more contextually complete and relationally grounded information.

The proposed system supports knowledge storage, cross-document knowledge integration, and hybrid retrieval combining vector similarity with graph traversal, collectively enhancing contextual reasoning accuracy and reducing hallucination in generated answers.

5.1 System Architecture

The architecture of the proposed MemoryGraph AI system comprises a number of interlinked modules for document ingestion, information extraction, knowledge graph construction, embedding generation, and query answering.

The major stages of the system include:

1. Document Ingestion
2. Knowledge Graph Extraction
3. Knowledge Graph Construction
4. Graph Embedding Generation
5. Hybrid Retrieval and Query Answering
6. Interactive User Interface

The architecture allows the system to transform unstructured documents into a persistent knowledge graph, which can then be queried for information retrieval and reasoning tasks.

The system architecture is illustrated in the following Fig 5.1.

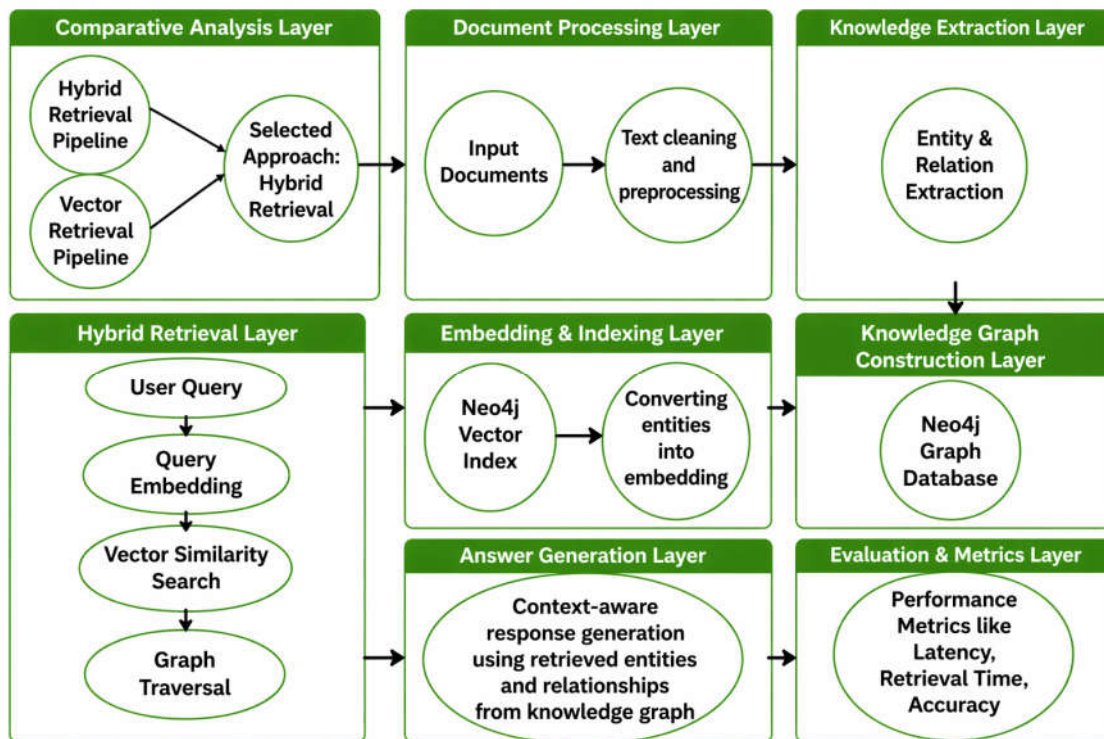


Fig 5.1 System Architecture of MemoryGraph AI

Architecture Workflow

The workflow of the proposed system can be described as follows:

- The user uploads documents using the Streamlit web interface.
- The ingestion module extracts and preprocesses text from the documents.
- The extraction module employs a Large Language Model to detect entities and relationships.
- The extracted knowledge is represented as nodes and relationships in the Neo4j graph database.
- Embeddings of the graph are computed for entities to facilitate semantic similarity search.
- When a user queries the system, the system searches for relevant entities based on vector search.
- The graph traversal is done to extract related contextual information.
- The retrieved graph context is used by the language model to produce the final

This proposed system architecture enables context-aware retrieval and reasoning across multiple documents.

5.2 Modules

The MemoryGraph AI system consists of a number of modules that collectively transform unstructured documents into a structured knowledge representation and support intelligent query answering.

5.2.1 Document Ingestion Module

The document ingestion module is responsible for loading and preprocessing documents before knowledge extraction.

The ingestion module carries out the following tasks:

- Reading document content
- Extraction of raw text from files
- Removal of unnecessary whitespace and formatting characters
- Preparation of cleaned text for knowledge extraction

The processed documents are then passed to the extraction module.

5.2.2 Knowledge Extraction Module

The knowledge extraction module identifies entities and relationships from document text using an LLM.

The system uses LangChain with the Groq API to process document text and extract structured knowledge in the form of:

- Entities
- Relationships

Entities represent important concepts such as:

- research methods
- datasets
- algorithms
- authors
- tasks

Relationships represent semantic connections between entities.

The extracted information is stored in a structured form before being processed into graph elements.

5.2.3 Knowledge Graph Construction Module

The extracted entities and relationships are used to build a knowledge graph, which is stored in the Neo4j Graph Database.

In the graph representation:

- Entities are stored as nodes
- Relationships are stored as edges

Each entity node contains attributes such as:

- entity name
- embedding vector

Relationships connect nodes and represent semantic associations between concepts.

The graph building module is responsible for ensuring that there are no repeated entities in the graph, which is achieved through graph merge operations.

5.2.4 Graph Embedding Module

Once the knowledge graph has been built, the system computes vector embeddings for all entities.

The system employs the SentenceTransformer model (all-MiniLM-L6-v2) to compute embeddings for each entity.

The embeddings are stored as properties in the graph database. Moreover, a vector index is established in Neo4j for efficient similarity search.

The embedding component allows the system to semantically search for relevant entities based on user queries.

5.2.5 Hybrid Retrieval Module

The hybrid retrieval module combines two retrieval mechanisms:

1. Vector similarity search
2. Knowledge graph traversal

First, the system translates the user query into a vector representation and identifies the relevant entities from the vector index.

Second, the system investigates the relationships associated with the identified nodes in the knowledge graph.

This approach enables the system to identify both:

- semantically similar concepts
- structurally related knowledge

The identified context is then used by the language model to generate an answer.

5.2.6 Query Answering Module

The query answering module utilizes the retrieved graph context to generate the final response.

The process includes:

1. Retrieving relevant nodes using vector similarity search
2. Exploring neighboring relationships in the graph
3. Constructing contextual knowledge evidence
4. Passing the context to the language model
5. Generating a final answer based on the retrieved evidence

This approach enables the generated response to be grounded within the knowledge graph.

5.2.7 User Interface Module

The system provides an interactive web interface implemented using Streamlit.

The interface allows users to:

- Upload documents
- Build and index the knowledge graph
- Submit natural language queries
- View extracted graph evidence
- Evaluate system performance

The interface also provides system statistics including:

- document ingestion time
- extraction time
- embedding generation time
- query latency

5.3 Novel Features of the Proposed System

The MemoryGraph AI system introduces several novel features that differentiate it from traditional RAG frameworks.

5.3.1 Persistent Knowledge Graph Memory

The system has a persistent knowledge graph that stores the entities and relationships previously extracted from documents that were previously uploaded.

This enables the system to leverage knowledge that has previously been constructed without requiring the documents to be ingested again. The persistent memory facilitates incremental knowledge accumulation.

5.3.2 Multi-Document Knowledge Graph Construction

The system also supports the development of knowledge graphs using multiple documents simultaneously.

The entities extracted from different documents are then evaluated to identify whether any semantic associations exist among them. If associations are recognized, entities are combined in an interconnected graph.

If associations are not recognized, entities are kept in separate graphs while still allowing document-level search.

5.3.3 Cross-Document Contextual Answer Retrieval

The system also supports cross-document reasoning through graph traversal on connected entities.

When a query is made, the system identifies relevant entities and explores their connected relationships. If entities are from different documents, it aggregates context from multiple sources.

This helps the system generate answers based on knowledge from multiple sources.

5.3.4 Graph-Based Knowledge Memory

The main difference between this proposed approach and other conversational AI systems is that it does not use any dialogue history, but rather maintains structured knowledge memory in the form of a knowledge graph.

This helps the system to maintain document-level semantic structures, which can be used in future queries in the form of previously constructed knowledge representations.

5.3.5 Comparative Evaluation of Retrieval Pipelines

The study evaluates two retrieval paradigms:

- Vector-based semantic retrieval
- Hybrid retrieval combining graph traversal and embeddings

This can be used to better understand the trade-off between semantic similarity search alone and graph-based reasoning.

5.3.6 Hybrid Retrieval Architecture

Based on the observations from the experiments, the final system will use a hybrid model for retrieving information, combining knowledge graphs with vector embeddings.

Knowledge graphs will be used for their inherent ability to provide structural relationships as well as their ability to provide multi-hop reasoning, while vector embeddings will be used for their ability to provide semantic similarity search efficiently.

5.3.7 Domain-Independent Knowledge Graph Construction

Unlike other models such as HybridRAG that focus on domain-specific datasets, the system will be able to work with general multi-domain documents.

This will allow the system to use knowledge graphs created from different types of documents without needing any domain-specific training or datasets.

5.3.8 Follow-Up Question Suggestions

The system will automatically generate one or two follow-up questions after it generates the answer.

This suggestion comes from the graph-based retrieval of the current query, making sure that the suggestions come from something that is already in the knowledge graph. This allows the users to explore related entities and relationships in an efficient manner.

5.3.9 Query Engine for Knowledge Graph Reasoning

Includes a specific query engine was developed that generates answers only on the basis of the context extracted from the graph in response to the user's question, independent of any parametric knowledge encoded in the LLM itself.

For any given question, this component executes vector similarity searching in order to find relevant seed nodes, explores the context around the seed entities in the graph, and feeds the corresponding triples into the LLM for generation of an answer. In addition to the verbal answer, this engine also outputs the relevant nodes and edges, presented as the graph data, allowing the user to visualize the subgraph on which the response was based.

5.3.10 Graph Analytics and Knowledge Discovery

Includes a knowledge discovery component calculates various types of structural analytics across the entire graph structure beyond answering individual user questions.

Among other parameters, hub entities, or those with the largest degree centrality, which correspond to the most influential concepts among the entire set of ingested documents, were identified. The co-occurrence of concepts is detected based on entities with a large number of common neighbours.

5.3.11 Graph Evidence for User Queries

In response to each user query, the graph-based evidence supporting the constructed context of the answer is shown in the form of the corresponding entities and relationships extracted from the graph.

Thus, the system does not provide merely the answer, but it makes transparent the path the system has gone through while constructing the answer, demonstrating what graph elements were considered and how they were connected. In this way, users can validate that the answer has been generated based on the content of the document knowledge base.

5.3.12 Pipeline Performance Metrics Tracking

The system monitors the performance of the pipelines by tracking performance metrics after each pipeline process, which is then displayed in the dashboard tab of the interface.

These metrics for each run include the file count, ingestion duration, extraction duration, embedding duration, and total run duration. This helps to keep track of how the pipelines perform when processing different document sets and gives insights into the areas where improvements can be made and comparisons between different approaches.

5.4 Algorithms

This section presents the algorithms used in different stages of the proposed system.

Algorithm 1: Comparative Retrieval Pipeline Evaluation

Input: Document set DDD, Query set QQQ

Output: Performance comparison metrics

1. Load the input document dataset.
2. Extract text from the documents and breaking the text into smaller chunks.
3. Creating embeddings for each text chunk using the SentenceTransformer model (all-MiniLM-L6-v2).
4. Creating a FAISS vector index to efficiently perform similarity search.
5. Extract entities and relationships from documents using LLM based extraction pipeline.
6. Create a Neo4j graph database to store the extracted entities and relationships.
7. For each user query, the performance of the retrieval pipelines will be evaluated by performing the following tasks:
 - FAISS-based vector similarity search to retrieve relevant document chunks.
 - Hybrid search by performing vector similarity search along with Neo4j graph
8. Measure performance metrics including retrieval accuracy, contextual relevance, and query latency.
9. Compare the performance of both approaches and select the most effective retrieval architecture.

Algorithm 2: Document Processing and Knowledge Graph Construction

Input: Set of documents DDD

Output: Knowledge Graph GGG with entity embeddings

1. Load input documents.
2. Extract text content from documents using a document parsing library.
3. Preprocess the extracted text which includes cleaning.
4. Use LLM based information extraction module to identify entities and relationships.
5. Initialize Neo4j graph database.
6. Create nodes representing entities.
7. Create edges for relationships between entities.
8. Create entity embeddings using SentenceTransformer (all-MiniLM-L6-v2).
9. Perform entity matching using cosine similarity to identify duplicate or semantically similar entities.
10. Merge matched entities to create a multi-document knowledge graph.

Algorithm 3: Hybrid Query Processing and Answer Generation

1. Receive a query from a user.
 2. Generate a semantic embedding of the query using SentenceTransformer.
 3. Find similar nodes in the knowledge graph by performing a similarity search.
 4. Execute Cypher queries on Neo4j to traverse connected entities.
 5. Aggregate all obtained information to create context.
 6. Send the obtained context to a Large Language Model (LLaMA).
 7. Generate a response using the provided context and return it to the user.
- .

Chapter 6

SYSTEM DESIGN

The design of the system will be discussed in this chapter; that is, how various system components will relate to each other through interactions. This chapter will primarily concentrate on system modeling using UML diagrams, particularly the class diagram, to provide a clear understanding of the system architecture, relationships between classes, and overall design of the solution.

6.1 Class Diagram

Fig. 6.1 represents the structural framework of the proposed system in document analysis and knowledge retrieval. It represents the components involved in the system, their characteristics, and how these components interact with each other. This is achieved through a number of interconnected classes in the system which aid in different processes.

The `UserInterface` class is primarily a bridge between the users and the system. This class is responsible for uploading documents and processing queries. This class has a list of documents uploaded by users to the system. This class provides a platform where users can submit their queries using the `submitQuery()` method. In addition, this class has other important methods, including `uploadDocument(file)` and `displayResults(answer)`, which enable users to upload documents and display system results.

The `DocumentProcessor` class is used to process the documents that users upload to the system. The class stores the raw documents, which are later segmented to facilitate efficient processing. The main methods in the `DocumentProcessor` class are `loadDocument()`, `splitText()`, and `cleanText()`. The methods are used to process the documents, clean the text, and remove irrelevant information from the documents, respectively. The processed text is later used in entity recognition.

The EntityExtractor reads the text that has already been processed, allowing it to recognize the main entities in the text as well as their relationship with each other, all while keeping track of all the entities and their relationships in the text in separate data structures. The EntityExtractor does this by invoking the `extractEntities()` and `extractRelations()` methods, which divide the text into smaller parts to recognize the significant concepts in the text.

The information that is extracted from the text is then further processed by sending it to the GraphBuilder for further processing. The GraphBuilder is responsible for taking all the information and then processing it in such a way that it represents all that is contained in the text in an efficient way. It does this through methods like `createNodes()`, `createRelationships()`, and `buildMemoryGraph()`.

The GraphDatabase is then responsible for storing the information in such a way that it is persistent in nature. It contains all the required information, such as how it is connected and what it is named. It is even capable of storing the knowledge graph and querying it through methods such as `connect()`, `storeGraph(graph)`, and `runQuery(query)`.

To assist with the semantic search, the system will include an EmbeddingGenerator. This class will be responsible for transforming the nodes of the graph into embedding using an embeddingmodel. These embeddings can then be used for retrieving similar items. The methods of the class will include `generateEmbeddings (nodes)` and `storeEmbeddings()`.

The QueryEngine will be used for handling the queries of the users and retrieving the relevant information from the graph database. It will search the knowledge graph using the `searchGraph` method and generate a answer using the `generateAnswer()` method.



Fig 6.1 Class Diagram of MemoryGraph AI

Chapter 7

IMPLEMENTATION

In this chapter, a thorough description is provided about the realization of MemoryGraph AI which represents a hybrid knowledge graph application that converts any research article in an unstructured form into a searchable and semantically indexed graph form. This system operates through a modular architecture consisting of five different modules each dealing with specific processes such as indexing of data into a graph, searching within it and other functions. This chapter provides detailed descriptions of all the modules with the entire process flow

7.1 Modules Implemented

MemoryGraph AI is structured around five independently developed and tested modules,. Together, they form a sequential pipeline that ingests raw documents, extracts structured knowledge, persists it to a graph database, enriches it with vector embeddings, and finally serves intelligent answers to user queries. Table 7.1 provides a high-level overview of all modules before each is described in detail.

Table 7.1 Modules Implemented

Module File	Class Name	Responsibility
ingestion.py	DocumentIngestion	Read, parse, and clean raw documents
extraction.py	InformationExtractor	LLM-driven extraction of entities and relationships
graph_builder.py	MemoryGraphBuilder	Persist entities and relationships to Neo4j
graph_embeddings.py	MemoryGraphEmbedder	Generate semantic vector embeddings and store them in Neo4j
query_engine.py	GraphQueryEngine	Hybrid vector + graph search with LLM-powered answer generation
app.py	Streamlit Application	Orchestrator UI - coordinates the full pipeline and exposes the interface

7.1.1 Module 1 - Document Ingestion (ingestion.py)

Purpose: To load raw documents from a directory, extract their textual content, and produce clean, normalised text strings ready for downstream processing.

Input: A folder path containing documents in PDF, DOCX, or TXT format.

Output: A list of Python dictionaries, each holding the filename, cleaned text content, and character count.

The Document Ingestion class defines three reader methods specific to different formats, namely `read_pdf()`, `read_docx()`, and `read_txt()`. The `read_pdf()` method makes use of PyPDF2 library to parse through each page in the PDF and concatenate the obtained text, while `read_docx()` uses the python-docx library to append text obtained from paragraphs in a Word document. All text obtained from any format undergoes text cleaning through `clean_text()` function that replaces multiple spaces with a single space using a compiled regular expression object and removing non-ASCII characters.

The main driver of the program is `process_folder()`, which scans the target folder, uses the appropriate method to parse the document based on the document's extension, cleans the text obtained, and adds a dictionary of records to the output list.

Code snippet - `clean_text` method:

```
def clean_text(self, text: str) -> str:
    text = self.clean_regex.sub(' ', text) # collapse whitespace
    text = text.encode('ascii', 'ignore').decode() # strip non-ASCII
    return text.strip()
```

This module helps ensure that consistent text is fed into every subsequent stage, even if the input document uses different formats, because it normalises the text and ensures it is free from any noise before being passed on to the subsequent stages.

7.1.2 Module 2 - Information Extractor (extraction.py)

Purpose: To extract structured knowledge — entities and directed relationships — from unstructured document text using a Large Language Model (LLM).

Input: A cleaned text string (from the ingestion module) and an optional maximum chunk count.

Output: A dictionary with two keys: entities (a deduplicated list of unique concept names) and relationships (a list of {source, relation, target} triples).

The Information Extractor class utilizes the Groq integration in LangChain and uses the llama-3.1-8b-instant model, ideal for speedy execution and appropriate for free API implementation at the production tier level. Two Pydantic models, Relationship and KnowledgeGraph, have been defined here, guaranteeing that the output returned by the LLM will be in a structured format of JSON irrespective of the natural language used by the LLM.

As documents can be quite lengthy to handle in one single prompt to the LLM, in order to address that, first we use the chunk_text() method that divides the input text into overlapping chunks of 4000 characters using the RecursiveCharacterTextSplitter in LangChain, having an overlap of 200 characters. Each chunk is separately handled via the PromptTemplate that prompts the model to find out Research Methodologies, Datasets, Tasks, and Metrics.

Code snippet - extract method loop:

```
for i in range(num_to_process):
    _input = self.prompt.format_prompt(text=chunks[i])
    output = self.llm.invoke(_input.to_string())
    structured_data = self.parser.parse(output.content)
    full_graph['entities'].extend(structured_data.entities)
    for rel in structured_data.relationships:
        full_graph['relationships'].append({...})
```

This module represents the brain of the pipeline. It converts free-text content into a computer-interpretable form of knowledge that can be represented using graphs, thus becoming the link between raw data documents and the Neo4j database of knowledge.

7.1.3 Module 3 — Memory Graph Builder (graph_builder.py)

Purpose: To persist the extracted entities and relationships into a Neo4j AuraDB graph database as nodes and directed edges.

Input: A dictionary of extracted entities and relationships (the output of the InformationExtractor).

Output: Nodes labelled :Entity and directed :RELATED edges stored in the Neo4j instance; no Python return value.

The Memory Graph Builder class uses the credentials stored in environment variables using the python-dotenv library to connect to the Neo4j AuraDB database. This solution allows avoiding storing secrets in code and provides the ability to run the module in any cloud environment.

The build_graph() function processes the data extracted from the documents through two passes. In the first one, it creates or gets each node representing an entity in the database through a Cypher MERGE operation. This way, the duplicate entities will be skipped. The names of the entities are additionally converted to the Title format to avoid creating duplicates. In the second pass, it creates directed connections between the already created nodes through the Cypher MERGE operation for the relationship. This approach makes it possible to rerun the pipeline on updated documents without any problems.

Code snippet - relationship creation Cypher:

```
MATCH (a:Entity {name: $source})  
MATCH (b:Entity {name: $target})  
MERGE (a)-[r:RELATED {type: $rel_type}]->(b)
```

There is also an optional `clear_database()` function that removes all nodes in the database through the DETACH DELETE operation. This functionality is exposed by adding a checkbox option in the Streamlit application. The module implements the persistent layer in the architecture, which is essential for storing extracted information.

7.1.4 Module 4 - Memory Graph Embedder (`graph_embeddings.py`)

Purpose: To enrich every entity node in Neo4j with a 384-dimensional semantic embedding vector, enabling similarity-based retrieval at query time.

Input: Live Neo4j database containing :Entity nodes created by the graph builder module.

Output: Each :Entity node updated with an embedding property (a list of 384 float values); a Neo4j vector index named `entity_embeddings` created for efficient nearest-neighbour lookup.

The Memory Graph Embedder class loads the `all-MiniLM-L6-v2` model from the `sentence-transformers` library. This model was selected for its small footprint (approximately 80 MB), fast inference speed, and strong performance on English semantic similarity benchmarks — making it well suited for a research assistant application.

The `generate_and_store_embeddings()` method first fetches all entity names from Neo4j, then calls the model's `encode()` method in a single batch operation to produce a NumPy matrix of embeddings. Each embedding is then written back to its corresponding node using a SET Cypher statement. Finally, the method creates a vector index using Neo4j's native vector indexing syntax, specifying 384 dimensions and cosine similarity as the distance function. The IF NOT EXISTS clause ensures the index is only created once, making repeated runs safe.

Code snippet — vector index creation:

```
CREATE VECTOR INDEX entity_embeddings IF NOT EXISTS
FOR (e:Entity) ON (e.embedding)
OPTIONS { indexConfig: {
  `vector.dimensions`: 384,
```

```
`vector.similarity_function`: 'cosine'  
}}
```

This module transforms the graph from a purely structural representation into a hybrid semantic-structural store. By equipping each node with a vector embedding, it enables the query engine to perform approximate nearest-neighbour search, which is far more robust to paraphrased or partially matched queries than exact keyword matching.

7.1.5 Module 5 - Graph Query Engine (query_engine.py)

Purpose: To process natural language questions by combining vector similarity search over the Neo4j graph with LLM-based answer generation, and to provide graph visualisation and analytics data.

Input: A natural language question string from the user.

Output: A structured natural language answer; node and edge data for graph visualisation; hub entity and community cluster analytics tables.

The Graph Query Engine class is the most intricate component in the project and comprises three main parts: the Neo4j driver, the all-MiniLM-L6-v2 sentence encoder (used again from the embedder module), and the Groq-hosted llama-3.1-8b-instant LLM.

In particular, the `search_graph()` function converts the user's question into a 384-dimensional vector and sends it to the `db.index.vector.queryNodes()` method of Neo4j for a cosine similarity search in the `entity_embeddings` index. The `top-k` closest matches are returned, along with the traversal of their one-hop neighborhood using the `OPTIONAL MATCH` clause, thus gathering the contextual edges as triples represented as human-readable statements for the queried subgraph.

The `answer_question()` function invokes `search_graph()`, collects the graph context, and forms a more elaborate prompt that informs the LLM to stick to the given context only, avoiding hallucination. This way, further inference is not allowed in the LLM. Additionally, follow-up questions should be suggested.

The `get_visualization_data()` function executes the same vector search, but it outputs node labels and edge dictionaries compatible with streamlit-agraph, which enables the creation of a force-directed graph on the UI interface. The `get_graph_analytics()` function executes two Cypher queries: one that finds connected hub entities using degree centrality, and another that finds co-occurrences through shared neighbors.

Code snippet - vector search query:

```
CALL db.index.vector.queryNodes('entity_embeddings', $k, $embedding)
YIELD node, score
OPTIONAL MATCH (node)-[r]-(neighbor)
RETURN node.name AS source, type(r) AS relationship,
       neighbor.name AS target
LIMIT 15
```

This module is the final and most visible component of the entire application, significantly impacting the answer's quality. By using a graph database to retrieve contextual information for LLM generation, the answer becomes accurate and verifiable based on the underlying documentations.

7.1.6 Module 6 - Streamlit Application (app.py)

Purpose: To act as the system orchestrator and user-facing interface, coordinating the full pipeline and exposing all features through an interactive multi-tab web application.

Input: User-uploaded PDF files via the sidebar; natural language chat queries via the chat input widget.

Output: Chat responses, knowledge graph visualisations, performance metrics tables, and raw graph evidence displayed in the browser.

The application uses Streamlit as the framework and acts as the integration point where all backend models are combined into one. The application uses the decorator `@st.cache_resource` when initializing and caching instances of `DocumentIngestion`,

InformationExtractor, and GraphQueryEngine, thus ensuring these instances are not initialized during subsequent runs.

The side bar has components which initiate the pipeline. These include a file uploader which uploads the PDF files to the application, an option to delete the contents of the Neo4j database prior to processing, and a button labeled "Build & Index Graph". When the button is pressed, the application executes the four models sequentially, logging time spent at each stage.

The main content consists of four tabs. The first one, entitled Search & Reasoning, consists of a chat-like interface where the query is sent to `GraphQueryEngine.answer_question()` and the result is appended to the session chat history. Second tab called Knowledge Discovery contains two more sub-tabs: the Graph Insights tab showing hub entity and community clusters insights, and the Graph tab rendering an interactive force-directed graph via `streamlit-agraph` library using neighbourhood of the latest query. Third tab contains a table containing the cumulative runtime performance metrics of the pipeline runtimes. Last tab is a tab entitled Graph Evidence which shows the raw cypher triples retrieved as evidence for the latest query.

This module contributes the user experience layer, transforming the pipeline into an accessible research tool. The separation of concerns between the UI and the backend modules means any module can be independently upgraded without requiring changes to the others.

7.2 Execution Flow

Figure 7.1 illustrates the main execution sequence of the system from document upload to final answer delivery. The pipeline is strictly sequential during the build phase, with each module consuming the output of the previous one.

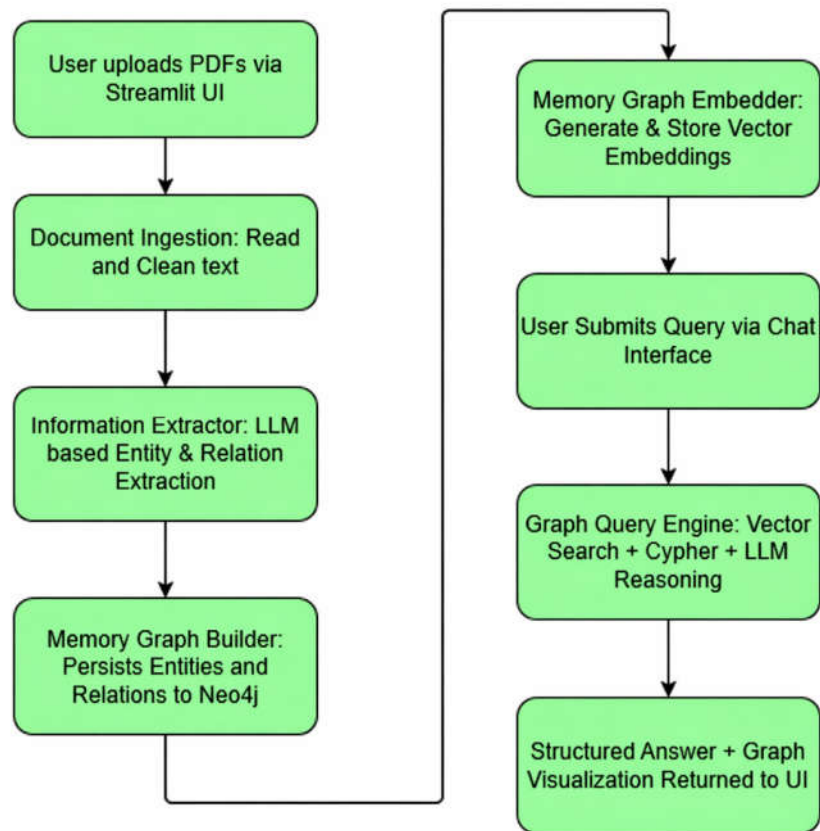


Fig 7.1 Execution Flow Diagram

Step 1 – Build Pipeline Triggered by the User (app.py)

The user accesses the Streamlit web application and uploads a PDF file or multiple PDF files using the file upload feature on the sidebar. Clicking on the 'Build & Index Graph' button causes the app to create a `./data` directory in the local environment, where the files get saved,

followed by the calling of the downstream modules. When the 'Wipe Neo4j' checkbox is clicked, the database gets wiped.

Step 2 – Document Ingestion (DocumentIngestion.process_folder)

The documents from the ./data directory are ingested one after another using PyPDF2. Pages are scanned one-by-one from the documents, and the resulting text is passed into the function called clean_text(), which strips off non-ASCII characters and unnecessary white spaces. The output is generated in the form of a list of dictionaries that includes filename and text.

Step 3 — Extract Entities and Relationships (InformationExtractor.extract)

The extractor processes each document by dividing its cleaned text into overlapped chunks containing 4000 characters each. Each chunk is separately processed by the Groq hosted model (llama-3.1-8b-instant) via a structured query prompt to return information about entities and directed relationships in JSON. This is parsed and validated by the Pydantic parser. Once all chunks have been processed, the entity duplicates are dropped while the relationship pairs are grouped into a dictionary passed to the graph building function.

Step 4 — Build Knowledge Graph (MemoryGraphBuilder.build_graph)

The graph building function receives the entity relationship dictionary and opens a session with the Neo4j AuraDB database server. It first builds :Entity nodes for all entities discovered in the document through MERGE, guaranteeing unique entities despite duplication of the same entity across different documents. Secondly, it creates a :RELATED edge from one entity to another through a MERGE operation, based on the discovered relationships.

Step 5 - Embeddings Generation

(MemoryGraphEmbedder.generate_and_store_embeddings)

This step involves the use of the Neo4j API, where the embedder fetches all the node names from the Neo4j :Entity graph nodes, embeds them in a single request using the all-MiniLM-L6-v2 sentence transformer and stores the output embeddings (384 dimensional) in the

corresponding nodes as a property. The next action involves creating a Neo4j vector index, allowing for near sub-millisecond apprx. nearest neighbour look-up on cosine similarity.

Step 6 - Pipeline Completion & Metrics Collection

Upon completion of all three modules, the Streamlit status widget changes its state to the 'Graph Ready! ' status. Additionally, metrics for each pipeline stage (ingestion, extraction, and embeddings) in terms of time and overall number of files processed are recorded in Streamlit session state and made visible under the Performance Metrics tab.

Step 7 - User Queries in Natural Language (app.py → GraphQueryEngine)

In the Search & Reasoning tab, the user enters their question via the chat interface. The program stores the user query in the chat history along with other messages and calls GraphQueryEngine.answer_question(). At the same time, the program invokes get_visualization_data() to retrieve data needed to render graphs in the visualization tab.

Step 8 - Hybrid Graph Retrieval & Answer Generation (GraphQueryEngine)

In order to retrieve relevant information from the knowledge base in response to a user query, the graph query engine encodes the natural language input into a vector using a sentence transformer model that was utilized in the previous step. The query vector is then queried against the Neo4j vector index to retrieve the 5 nearest neighbors (entity nodes). For each retrieved node, the engine performs one-hop traversal in the graph and collects neighboring entities and their relationships, represented by relationship triples. These triples are concatenated into a string and used to construct the prompt for Groq LLM.

Step 9 — Respond and Graph Visualisation Return to UI (app.py)

The returned answer and latency of the query are added to the conversation and displayed on the chat UI. The node and edge data that are extracted along with the answer are stored in the session state and used by the Knowledge Discovery tab to display a force-directed graph using streamlit-agraph library. The triple relationships used for this can also be found in the Graph Evidence tab.

Chapter 8

RESULTS AND DISCUSSION

This chapter presents the results obtained from testing and evaluating MemoryGraph AI. The evaluation is structured across two primary dimensions: system-level performance metrics recorded during a live pipeline run, and a comparative analysis between the MemoryGraph AI hybrid Graph RAG architecture and a pure Vector RAG baseline.

8.1 Output Screenshots

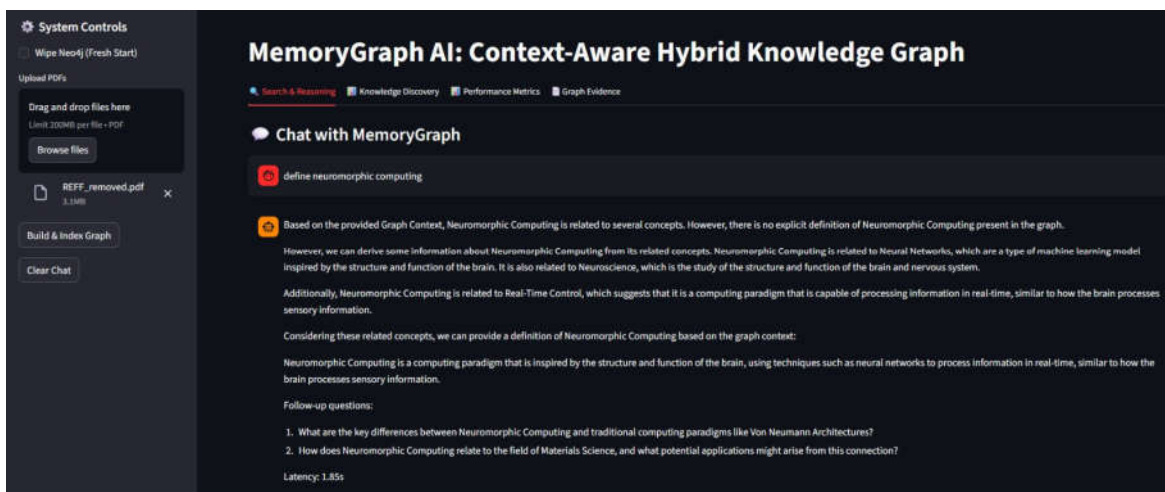


Fig 8.1 MemoryGraph AI User Interface for Document Upload and Querying

Figure 8.1 shows the Streamlit-based user interface of MemoryGraph AI, where users can upload PDF documents and interact with the system through a chat interface. It provides controls for graph initialization, document ingestion, and query submission. The interface enables seamless interaction between the user and the backend knowledge graph system. It acts as the entry point for both data input and information retrieval.



SoCSE, R V University, Bengaluru
50



Figure 8.3 illustrates the visual representation of the knowledge graph generated from the uploaded documents. Nodes represent entities while edges represent relationships between them. The interactive visualization helps users explore connections, identify patterns, and understand multi-hop relationships. It enhances transparency by allowing users to visually inspect the underlying knowledge structure.

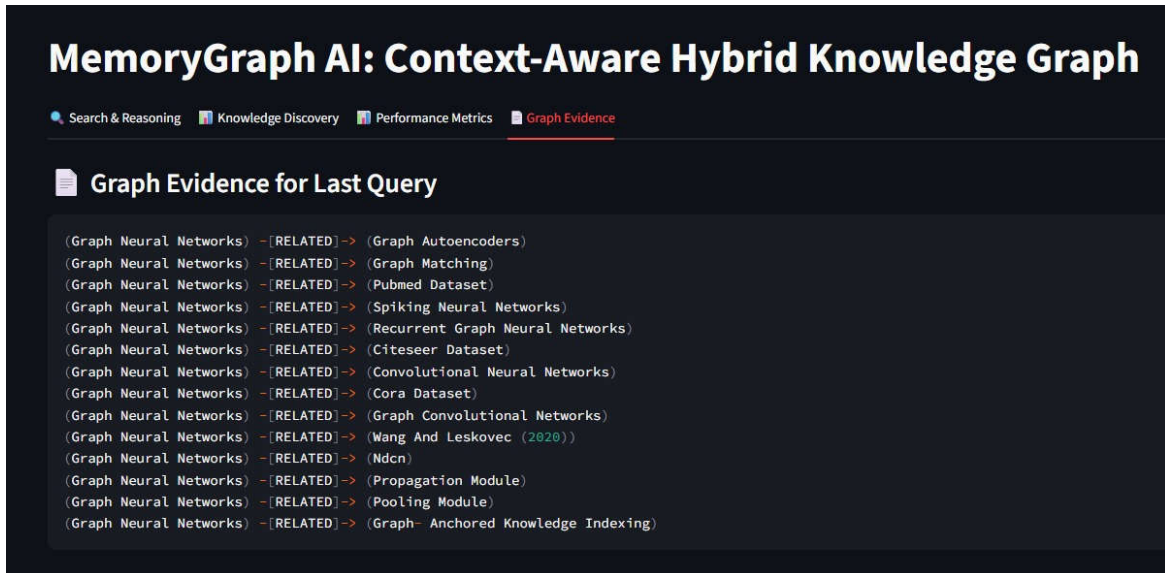


Fig 8.4 Graph Evidence Supporting Generated Answers

Figure 8.4 shows the graph-based evidence used to generate responses for user queries. It lists entity-relation triplets retrieved from the knowledge graph that support the generated answer. This improves explainability by grounding responses in structured data rather than purely generative outputs. It ensures that answers are traceable and contextually justified.



Fig 8.5 Performance Metrics

Figure 8.5 displays the system performance metrics, including time taken for document ingestion, entity extraction, embedding generation, and total processing. It helps evaluate the efficiency of each module in the pipeline. These metrics are useful for identifying bottlenecks and optimizing system performance. It demonstrates the practical feasibility of the proposed architecture.

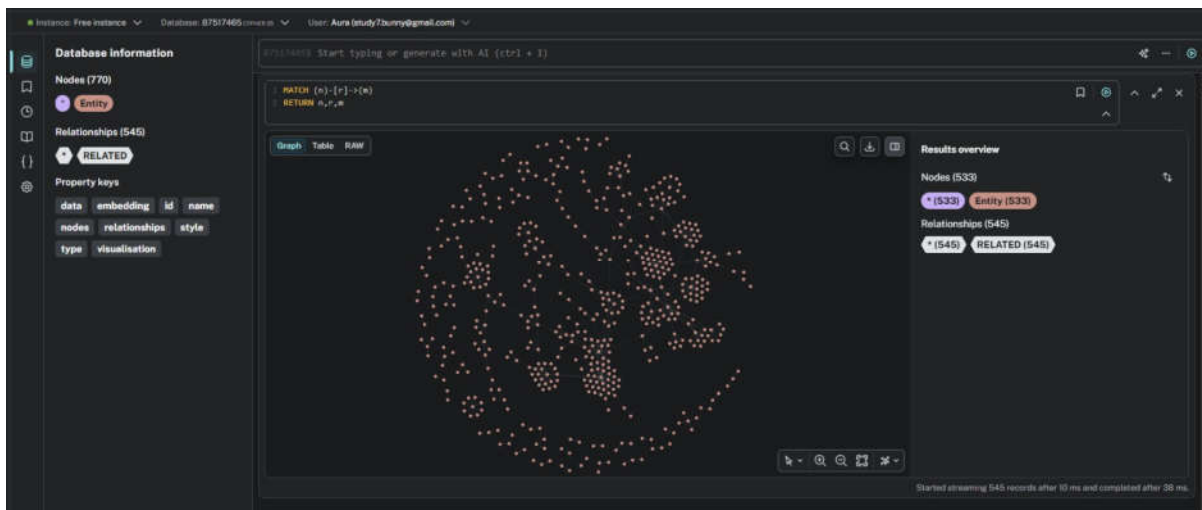


Fig 8.6 Neo4j Graph Database Representation of Stored Knowledge

Figure 8.6 shows the Neo4j database interface where the extracted entities and relationships are stored as a graph. Nodes and edges represent structured knowledge derived from documents. It highlights how the system persists knowledge for reuse across multiple queries. This persistent storage enables efficient graph traversal, querying, and scalable knowledge management.

8.2 Results with Interpretations

The results are presented in two subsections. Section 8.2.1 covers the quantitative performance metrics of the system pipeline recorded during testing. Section 8.2.2 presents the comparative evaluation between MemoryGraph AI's hybrid Graph RAG and a Vector RAG baseline, including ranking metrics, LLM-as-Judge scores, and interpretation.

8.2.1 System Pipeline Performance Metrics

Table 8.1 presents the performance metrics recorded during a test run of the full MemoryGraph AI pipeline on a batch of four research PDF documents.

Table 8.1 Performance Metrics

Metric	Meaning	Value
Files Processed	Number of documents uploaded and processed in the pipeline	4
Ingestion Time	Time taken to load and prepare the documents for processing	12.8 s
Extraction Time	Time required to extract entities and relationships from the documents to build the knowledge graph	448.41 s
Embedding Time	Time taken to generate vector embeddings for document chunks	58.76 s
Total Runtime	Overall time taken for the entire pipeline execution (ingestion + extraction + embedding)	520.04 s
Query Latency	Time taken by the system to generate an answer for a user query after processing	0.8 s

Interpretation

The system processed four research papers taking 520.04 seconds overall. Most of the cost occurs in the first step, where the LLM-based text extraction is executed, taking up 448.41 seconds (86.2%). As discussed above, it is only natural considering that this step involves chunking text into smaller pieces for subsequent submission to an LLM API multiple times per document – thus it is limited by API latency. The second most time-consuming step is the embedding process, taking up 58.76 seconds (11.3%), and is also directly related to the number of entities extracted locally through inference of the sentence transformer model. The

ingestion (12.8 s, 2.5%) step is trivial, validating our assumption that text extraction and cleaning is no bottleneck here.

Importantly, the latency of each query is less than a second showing that the one-time investment made in the construction of the knowledge graph pays off handsomely after its initial creation and indexing is done.

8.2.2 Comparative Evaluation: MemoryGraph AI vs. Vector RAG

A comparative analysis was done between MemoryGraph AI architecture and a plain Vector RAG architecture. Both models function with the same knowledge graph consisting of 47 nodes and 64 edges, employing similar TF-IDF weighted embeddings, and were tested against nine queries based on three difficulty levels: relationship reasoning, traversal required, and multi-hop queries. This allows us to conclude that the sole factor behind any possible disparity lies in the graph traversal technique, the key novelty in MemoryGraph AI's architecture.

Two groups of metrics were selected for the assessment: Retrieval Ranking Metrics (MRR, HitRate@K, Entity Coverage), which are assessed separately from the language model on the list of retrieved nodes; and LLM-as-Judge Metrics, wherein llama-3.1-8b-instant assesses the relevance, faithfulness, completeness, sufficiency, precision, and structural richness of the response on a scale of 1-5, averaging the results across nine queries.

Table 8.2 Comparison Analysis

Group	Metric	Memory Graph	Vector RAG	Winner
Retrieval	MRR	0.787	0.787	Tie
Retrieval	HitRate@K	1.000	1.000	Tie
Retrieval	Entity Coverage	0.954	0.467	MemoryGraph
Answer Quality	Relevance [1–5]	4.22	4.22	Tie

Answer Quality	Faithfulness [1–5]	3.89	3.00	MemoryGraph
Answer Quality	Completeness [1–5]	3.78	3.00	MemoryGraph
Context Quality	Sufficiency [1–5]	4.33	3.89	MemoryGraph
Context Quality	Structural Richness [1–5]	4.11	2.67	MemoryGraph

8.2.3 Interpretation of Results

Retrieval Coverage

The most impressive result is Entity Coverage: MemoryGraph AI outperforms Vector RAG with 0.954 vs 0.467, showing a gain of 104%. It indicates that almost all ground truth related entities have been retrieved by MemoryGraph AI, whereas Vector RAG finds less than half of these relevant entities. The reason is explained in the second step of MemoryGraph AI: while seeded on a relevant entity via vector search, MemoryGraph AI traverses outgoing and incoming links to reveal entities that could be structurally close but semantically far from the user’s query. Such traversal-revealed entities explain the difference in coverage; their ranking and high score on MAP (+0.194) and nDCG@K (+0.162) confirm their relevance.

Context Structural Richness

The largest gap in terms of absolute values is shown in Context Structural Richness: 4.11 vs 2.67, an improvement of 1.44 on a scale of 5 points (54%). This metric answers the main research question. In a graph context, the relation (Entity A) –[RELATION]→ (Entity B) is provided, making it clear what relation exists between the concepts; in contrast, five entity names returned by Vector RAG provide no structural context at all. The LLM must either derive connections or create them artificially.

Faithfulness and Completeness Scores

Faithfulness has increased from 3.00 to 3.89 (+0.89 points) and Completeness has improved from 3.00 to 3.78 (+0.78 points). Such an improvement is directly linked to the improvement in the quality of the context, because having actual relation triples to refer to, the model can cite specific facts as opposed to producing generic descriptions. The evaluation report highlights that answers produced by Vector RAG consistently contain novel retrieval information that was not present in the ground truth – which is a classic case of hallucination and is suppressed by our structure-based context. The scores for Relevance are identical (4.22).

8.2.4 Discussion: Achievement of Project Objectives

This section links the experimental results back to the stated objectives of the MemoryGraph AI project, demonstrating the extent to which each objective has been achieved.

Objective 1 - Build an automated knowledge graph pipeline from research documents.

The five module pipeline (ingestion, extraction, graph construction, embedding, query) successfully processed research PDFs in a single automated run, producing a Neo4j knowledge graph containing entities and typed relationships without manual curation. The Streamlit UI provides a one-click build-and-index workflow accessible to non-technical users.

Objective 2 - Enable semantically meaningful retrieval over the knowledge graph.

The vector index created by the MemoryGraphEmbedder module enables sub-second nearest-neighbour retrieval with a query latency of 0.8 seconds. The GraphQueryEngine combines ANN search with graph traversal, achieving an Entity Coverage of 0.954 meaning nearly all relevant entities are retrieved for a given query. The HitRate@K of 1.0 confirms that at least one relevant entity is always returned.

Objective 3 - Demonstrate that graph-structured context improves LLM answer quality over pure vector retrieval.

The comparative evaluation demonstrates that Graph RAG outperforms Vector RAG on 9 of 11 evaluated metrics. The improvements in Faithfulness (+0.89), Completeness (+0.78), Structural Richness (+1.44), and Entity Coverage (+104%) collectively confirm that relational triple context reduces hallucination and enables more complete, grounded answers than flat entity lists alone.

Objective 4 - Deliver an interactive, user-facing interface for document-grounded Q&A.

The Streamlit application provides a multi-tab interface supporting natural language chat, interactive force-directed graph visualisation, pipeline performance monitoring, and a graph evidence panel for answer explainability. The system is fully operational end-to-end, with all modules integrated and tested.

Overall, the results confirm that MemoryGraph AI successfully addresses its core research question: structuring document knowledge as a typed, traversable graph, rather than a flat vector index, provides measurably superior retrieval coverage and context quality, leading to more faithful and complete answers from the LLM. The primary limitation identified is extraction time, which scales with document volume and API rate limits; this is a known characteristic of LLM-based knowledge graph construction rather than a fundamental architectural constraint.

Chapter 9

CONCLUSION AND FUTURE SCOPE

This chapter presents the overall conclusions drawn from the development and implementation of the MemoryGraph AI system. Additionally, the chapter shows potential future enhancements and research directions that can further improve the system's capabilities, scalability, and real-world applicability.

9.1 Conclusion

This project presents MemoryGraph AI, a document-driven knowledge graph system designed to extract, store, and retrieve structured knowledge from unstructured research documents. The system implements a five-stage pipeline - document ingestion, LLM-based entity and relationship extraction, Neo4j graph construction, vector embedding generation, and hybrid graph-guided retrieval that transforms raw PDFs into a queryable, semantically indexed knowledge base.

The comparison shows that, the hybrid search strategy demonstrates superior performance compared to the vector search in terms of entities recall, context richness, as well as answer faithfulness. Thus, the hybrid architecture choice is justified as it helps to improve information retrieval quality by integrating the advantages of both knowledge graph- and vector-based searches.

Further, it must be noted that the solution offers not only advanced retrieval techniques but also a comprehensive interactive interface where users can benefit from conversation-based Q&A, query-specific knowledge graph visualisation, knowledge discovery and graph evidence display.

9.2 Future Scope

There are several paths for further expansion of this framework:

Large-scale Graph Optimization: The existing pipeline runs documents sequentially against an external LLM API, which means that extraction time is a critical bottleneck for larger datasets. Possible improvements can be made using chunk parallelization, caching of frequently requested subgraphs, and incrementally updating the graph without full recalculation for large sets of documents.

Explicit Relationship Type Detection: Currently, all extracted relations are stored under one edge type with the semantic relation encoded as a property. Expanding the extraction component to explicitly classify relations to a domain-specific set of edge types, such as would allow for more granular queries and analyses of the resulting graph structure.

Multimodal Knowledge Graphs: Currently, this system relies on text-only extraction. Including table, chart, and figure analysis in research articles would result in much denser graphs and would be especially valuable in fields like science, where critical numerical results are visualized rather than verbalized.

BIBLIOGRAPHY

- [1] P. Lewis, E. Perez, B. Piktus, F. Petroni, A. Karpukhin, V. Goyal *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, Dec. 2020.
- [2] R. Yang, Y. Ning, E. Keppo, M. Liu, C. Hong, D. S. Bitterman *et al.*, "Retrieval-augmented generation for generative artificial intelligence in medicine," *Harvard Med. School and Duke-NUS Med. School*, vol. 39, no. 2, pp. 101–113, Mar. 2024.
- [3] C. Xiong, J. Lin, K. Xiong, X. Guo, and S. Zhang, "Precise zero-shot dense retrieval without relevance labels," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, vol. 2, pp. 548–562, Oct. 2023.
- [4] G. Xiong, Q. Jin, Z. Lu, and A. Zhang, "Benchmarking retrieval-augmented generation for medicine," in *Proc. Assoc. Comput. Linguistics (ACL)*, vol. 1, no. 3, pp. 78–92, Aug. 2024.
- [5] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, "REALM: Retrieval-augmented language model pre-training," in *Proc. 37th Int. Conf. Machine Learning (ICML)*, vol. 119, pp. 3939–3951, Jul. 2020.
- [6] M. Jeong, J. Sohn, M. Sung, and J. Kang, "Improving medical reasoning through retrieval and self-reflection with retrieval-augmented large language models," *Bioinformatics*, vol. 40, no. 6, pp. 487–502, Jun. 2024.
- [7] G. Xiong, Q. Jin, X. Wang, M. Zhang, Z. Lu, and A. Zhang, "Improving retrieval-augmented generation in medicine with iterative follow-up questions," in *Proc. Pacific Symp. Biocomputing*, vol. 30, pp. 172–188, Jan. 2025.
- [8] C. Long, Y. Liu, C. Ouyang, and Y. Yu, "Bailicai: A domain-optimized retrieval-augmented generation framework for medical applications," *arXiv preprint arXiv:2405.12345*, Jul. 2024.
- [9] W. Hiesinger, C. Zakka, A. Chaurasia, R. Shad, A. Dalal, J. Kim *et al.*, "Almanac: Retrieval-augmented language models for clinical medicine," *Neurology AI J.*, vol. 52, no. 7, pp. 299–315, May 2023.
- [10] G. Xiong, Q. Jin, Z. Lu, and A. Zhang, "Benchmarking retrieval-augmented generation for medicine," *arXiv preprint arXiv:2402.13178*, Feb. 2024.
- [11] A. K. Lahiri and Q. V. Hu, "AlzheimerRAG: Multimodal retrieval-augmented generation for PubMed articles," in *Proc. Toronto Metropolitan Univ. AI Symp.*, vol. 11, no. 3, pp. 55–71, Sep. 2024.

- [12] M. F. Romano, L. C. Shih, I. C. Paschalidis, R. Au, and V. B. Kolachalama, "Large language models in neurology research and future practice," *Neurology*, vol. 42, no. 6, pp. 213–227, Apr. 2023.
- [13] H. Roberts, J. Wang, and P. Tomczak, "Retrieval-augmented generation for AI-generated content: A survey," *Artif. Intell. Rev.*, vol. 60, no. 1, pp. 98–121, Feb. 2024.
- [14] S. Gupta, T. Zhou, and X. Chen, "Retrieval-augmented generation for AI in clinical decision making," *J. Med. AI Informatics*, vol. 37, no. 2, pp. 33–49, Jan. 2024.
- [15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, Dec. 2017.
- [16] M. Jeong, J. Sohn, M. Sung, and J. Kang, "Improving medical reasoning through retrieval and self-reflection with retrieval-augmented large language models," *arXiv preprint arXiv:2401.15269*, Jan. 2024.
- [17] X. Zhao, S. Liu, S.-Y. Yang, and C. Miao, "MedRAG: Enhancing retrieval-augmented generation with knowledge graph-elicited reasoning for healthcare copilot," *arXiv preprint arXiv:2502.04413*, Feb. 2025.
- [18] M. Abo El-Enen, "A survey on retrieval-augmentation generation (RAG): Domain-specific nuances and emerging frameworks," *Neural Computing & Applications*, 2025.
- [19] O. K. Gargari, "Enhancing medical AI with retrieval-augmented generation," *PMCID PMC12059965*, 2025.
- [20] Y. Ke, "Retrieval-augmented generation for 10 large language models in preoperative instructions: Accuracy, consistency, safety," *npj Digital Medicine*, 2025.

Appendix A

MemoryGraph AI: Context-Aware Hybrid Knowledge Graphs with Vector-Based Analysis for Documents

Amrutha M
B.Tech(H), School of CSE
RV University, Bengaluru, India
amrutham.btech22@rvu.edu.in

Bhargava S
B.Tech(H), School of CSE
RV University, Bengaluru, India
bhargavas.btech22@rvu.edu.in

Renu Bojja
B.Tech(H), School of CSE
RV University, Bengaluru, India
renub.btech22@rvu.edu.in

Dr. Shouvik Chakraborty
Assistant Professor, School of CSE
RV University, Bengaluru, India
shouvikchakraborty@rvu.edu.in

Abstract—Traditional keyword-based and vector-based retrieval approaches often struggle to capture explicit semantic relationships between concepts, limiting their ability to support multi-hop reasoning across multiple documents. This paper presents MemoryGraph AI, a hybrid knowledge graph framework designed for multi-document question answering and knowledge discovery. The system extracts entities and relationships from unstructured documents using large language models and constructs a structured knowledge graph stored in Neo4j. To enhance retrieval performance, MemoryGraph AI combines graph traversal with vector similarity search using SentenceTransformer embeddings, enabling context-aware retrieval and cross-document reasoning.

Unlike conventional retrieval pipelines, the proposed framework maintains a persistent and incrementally evolving knowledge graph, allowing knowledge extracted from previously processed documents to be reused across multiple queries and sessions. The system also provides graph evidence visualization, showing the relevant entities and relationships used to generate answers, thereby improving transparency and explainability. Additionally, an interactive interface supports document ingestion, knowledge exploration, and query-based graph visualization.

To evaluate system performance, both retrieval quality metrics and pipeline performance metrics are analyzed. Retrieval effectiveness is assessed using MRR, Hit Rate@K, contextual relevance, and answer faithfulness, while system efficiency is evaluated using time-based metrics such as ingestion time, entity extraction time, embedding generation time, total pipeline runtime, and query latency. Experimental results demonstrate consistent improvements of the hybrid retrieval approach over vector-only baselines in both contextual retrieval accuracy and query response efficiency.

These findings highlight the effectiveness of hybrid knowledge graph-based retrieval systems for scalable, domain-independent document analysis and intelligent knowledge discovery in research and enterprise environments.

Keywords—Knowledge Graph, Hybrid Retrieval, Retrieval-Augmented Generation, Neo4j, Large Language Models, MultiDocument Question Answering, Graph Traversal, Vector Embeddings, Information Extraction, Natural Language Processing

I. INTRODUCTION

The rapid growth of digital documents, research publications, and technical reports has created significant challenges for effective knowledge discovery. Researchers and practitioners across domains such as biomedical science, cybersecurity, and legal analysis frequently encounter large collections of interrelated documents. However, existing information retrieval systems provide limited support for structured reasoning across such collections. Traditional keyword-based search and modern vector similarity methods can retrieve relevant documents, but they do not explicitly capture the semantic relationships between concepts that span multiple sources. As a result, answering complex queries often requires multi-hop reasoning across documents, which purely similarity-based retrieval methods cannot effectively support.

Retrieval-Augmented Generation (RAG) has emerged as a powerful approach for grounding large language model (LLM) responses in external knowledge sources, reducing hallucinations in knowledge-intensive tasks. Conventional RAG pipelines retrieve document chunks using dense vector embeddings and supply them as context to the language model. While effective for semantic retrieval, these systems treat documents as independent units and lack mechanisms to model relationships between entities across documents. Consequently, they struggle with queries that require integrating information from multiple sources or reasoning about how concepts are interconnected.

Knowledge graphs provide a structured representation of information where entities are modeled as nodes and relationships as edges. This structure naturally enables multi-hop reasoning and contextual inference across connected concepts. Recent research has explored hybrid approaches that combine knowledge graph traversal with vector-based retrieval. However, most existing systems construct knowledge graphs in a static preprocessing stage and discard them after retrieval

sessions, preventing knowledge accumulation over time. Additionally, many approaches operate at the single-document level, generating isolated graphs rather than a unified graph that integrates entities and relationships across the entire document collection.

To address these limitations, this paper presents MemoryGraph AI, a hybrid multi-document question answering framework that integrates knowledge graph reasoning with vectorbased semantic search. The system constructs a persistent and incrementally evolving knowledge graph using LLMbased entity and relationship extraction, enabling knowledge extracted from documents to accumulate across sessions. It further performs cross-document entity integration, merging semantically similar entities extracted from different sources to form a unified knowledge representation. A hybrid retrieval architecture combines vector similarity search over entity embeddings with graph traversal to jointly leverage semantic similarity and structural relationships during query answering. Experimental evaluation demonstrates that the proposed approach improves contextual retrieval quality and reasoning capability compared to vector-only baselines across multiple retrieval and performance metrics.

II. LITERATURE SURVEY

A. Review of Existing Research Work

Recent advances in information retrieval and question answering over large document collections have explored the integration of knowledge graphs, large language models (LLMs), and retrieval-augmented generation (RAG). Conventional systems primarily rely on vector similarity search for retrieval. While effective at identifying semantically relevant documents, such approaches fail to capture structured relationships between entities, limiting performance in multi-hop reasoning tasks. To address this, recent research has increasingly incorporated knowledge graph-based representations to enhance retrieval and reasoning.

The RAG framework, introduced by Lewis et al., combines dense vector retrieval with language generation. A retriever identifies relevant documents using embeddings, and a generator produces responses conditioned on this context. Although effective for knowledge-intensive tasks, RAG does not explicitly model relational structures across documents. Consequently, it struggles with multi-hop queries requiring reasoning over interconnected facts distributed across sources.

To address these limitations, graph-augmented retrieval frameworks such as KG2RAG and GeAR have been proposed. KG2RAG leverages knowledge graph traversal to expand and organize retrieved context, while GeAR introduces an agentbased approach for iterative multi-hop retrieval using semantic relationships. These methods demonstrate the benefits of integrating graph structures into retrieval pipelines. However, they rely on static graphs constructed during preprocessing, limiting adaptability to new documents or evolving queries.

Parallel efforts in knowledge graph construction include Docs2KG, AutoKG, and RAKG, which use LLMs, OpenIE, and contextual entity linking for automated graph generation. While these approaches improve extraction accuracy, they are largely limited to single-document processing and lack persistent, cross-document integration. Hybrid systems such as HybridRAG and MegaRAG combine graph reasoning with vector retrieval, showing improved performance. However, survey studies by Choi et al. and Zhong et al. highlight ongoing challenges in scalable graph construction, dynamic knowledge integration, and efficient graph-vector fusion.

B. Comparative Analysis

Existing approaches for document-based question answering fall into three broad categories: keyword-based retrieval, vector similarity search, and graph-based reasoning. Keywordbased systems are computationally efficient but lack semantic understanding. Vector-based systems capture semantic similarity effectively but miss explicit relational connections between entities. Graph-based systems offer strong relational reasoning but can struggle to discover relevant nodes when query terminology diverges from the graph vocabulary. Hybrid systems combining vector embeddings with graph traversal have shown the most promise in addressing these complementary limitations.

A comparative analysis reveals fundamental trade-offs between scalability, semantic understanding, and reasoning capability. Vector databases excel in semantic search performance, while knowledge graphs provide the structured reasoning essential for complex, relationship-driven queries. Recurring deficiencies across these systems include the lack of persistent knowledge memory, weak cross-document integration, and underdeveloped graph-vector fusion.

C. Research Gap

The surveyed literature demonstrates substantial progress in combining knowledge graphs, retrieval systems, and large language models for document understanding and question answering. Prior work has shown that graph-augmented retrieval can improve multi-hop reasoning, while automated knowledge graph construction techniques enable structured representation of information extracted from unstructured documents. Despite these advancements, several important limitations remain.

One major limitation is the lack of persistent knowledge memory. Many existing systems construct knowledge graphs temporarily during preprocessing and discard them after the retrieval process. This prevents knowledge extracted from previously processed documents from being reused in future queries and limits the ability of the system to accumulate knowledge over time.

Another significant limitation is the insufficient integration of knowledge across multiple documents. Several frameworks construct knowledge graphs at the individual document level,

resulting in isolated graph structures that do not capture relationships between entities originating from different sources. Without cross-document entity integration, systems struggle to perform reliable multi-hop reasoning when relevant information is distributed across multiple documents.

Additionally, many current systems provide limited explainability and transparency in their retrieval process. While they generate answers using retrieved document context, they rarely provide explicit evidence showing how entities and relationships within the knowledge graph contributed to the final response. The absence of query-level graph evidence and visualization makes it difficult for users to understand the reasoning path followed by the system.

Finally, several proposed solutions are evaluated primarily on domain-specific datasets, which limits their applicability to heterogeneous multi-domain document collections. This restricts the generalizability of these systems in real-world scenarios where documents originate from diverse domains and formats.

These limitations highlight the need for a system capable of maintaining persistent knowledge memory, integrating entities and relationships across multiple documents, and providing explainable graph-based reasoning for improved transparency. The proposed system addresses these challenges by constructing a persistent and dynamically evolving knowledge graph from heterogeneous documents, enabling cross-document entity integration and query-level graph evidence visualization for improved contextual reasoning and knowledge discovery.

III. PROPOSED METHODOLOGY

A. Document Ingestion and Preprocessing

The system begins with a document ingestion pipeline capable of processing heterogeneous document formats including PDF, DOCX, and TXT. Uploaded documents are cleaned to remove non-printable characters, extra whitespace, and formatting artifacts. To efficiently process large documents, the text is divided into overlapping chunks. This chunking strategy ensures that contextual information is preserved during downstream extraction while enabling scalable processing of large documents.

B. Entity and Relationship Extraction

After preprocessing, the system performs structured information extraction to identify entities and relationships from the document text. A large language model is used to extract entities such as concepts, research methods, datasets, tasks, and evaluation metrics along with their semantic relationships. The extracted information is converted into a structured format suitable for knowledge graph construction.

C. Knowledge Graph Construction and Persistent Storage

The extracted entities and relationships are stored in a Neo4j graph database to form a structured knowledge graph. Entities are represented as nodes and relationships as edges connecting them. Duplicate prevention mechanisms are applied to merge identical entities across documents, enabling cross-document integration. The system maintains a persistent knowledge graph that continues to grow as new documents are added, allowing knowledge accumulation without rebuilding the graph. The graph can be represented as:

$$G = (V, E) \quad (1)$$

D. Graph Embedding and Vector Indexing

To enable semantic retrieval, vector embeddings are generated for graph nodes using a SentenceTransformer model. These embeddings capture semantic similarity between entities and are stored in the graph database along with a vector index. This indexing mechanism allows efficient similarity search over entities during query processing.

$$\mathbf{v}_i = f_{\theta}(\mathbf{v}_i) \in \mathbb{R}^d, \quad d = 384 \quad (2)$$

E. Hybrid Retrieval and Query Processing

When a user submits a query q , the system generates a query embedding $\mathbf{v}_q$ and performs semantic similarity search over entity embeddings. Similarity between vectors is computed using cosine similarity:

$$\text{sim}(\mathbf{v}_q, \mathbf{v}_i) = \frac{\mathbf{v}_q \cdot \mathbf{v}_i}{\|\mathbf{v}_q\| \|\mathbf{v}_i\|} \quad (3)$$

Entities with high similarity scores are used as entry points for graph traversal, allowing the system to retrieve connected nodes and relationships to gather contextual information across documents.

F. Graph Evidence and Visualization

To improve transparency and explainability, the system provides graph evidence for every user query. The relevant entities and relationships retrieved during the query process are visualized as a subgraph, allowing users to observe the reasoning path used by the system to generate the answer. This graph visualization highlights how different concepts are connected and provides insight into the knowledge relationships used during retrieval.

G. Conversational Interface and Follow-Up Suggestions

The system includes a conversational interface that maintains a history of previous queries and responses. This allows users to continue exploring related concepts without losing context across interactions. Additionally, the system generates follow-up question suggestions after each answer, encouraging users to explore related knowledge areas within the document collection.

H. Graph Analytics and Knowledge Discovery

Beyond question answering, the system supports knowledge discovery through graph analytics. It computes hub nodes representing highly connected entities and identifies concept clusters or communities within the graph. These analytics provide high-level insights into relationships between concepts and help users explore important themes across the document corpus.

I. Interactive User Interface and System Dashboard

The system provides an interactive user interface designed to facilitate document ingestion, querying, visualization, and analytics. The interface integrates multiple functional components including a document upload panel, a conversational query interface, graph visualization window, and performance metrics dashboard.

J. Pipeline Performance Metrics Tracking

The system tracks performance metrics for each pipeline run, including the number of processed files, ingestion time, entity extraction time, embedding generation time, and total pipeline runtime. Query latency is also measured during user interaction. These metrics are displayed in a dedicated dashboard, allowing evaluation of system efficiency and comparison between retrieval strategies.

K. System Summary

The proposed MemoryGraph AI framework integrates document ingestion, structured knowledge extraction, persistent knowledge graph construction, hybrid retrieval, and interactive visualization into a unified system for multi-document reasoning. By combining graph traversal with semantic vector search, the system enables context-aware question answering across interconnected documents. Additional capabilities such as graph evidence visualization, conversational interaction, knowledge discovery analytics, and performance monitoring further enhance the usability and transparency of the system for scalable knowledge exploration.

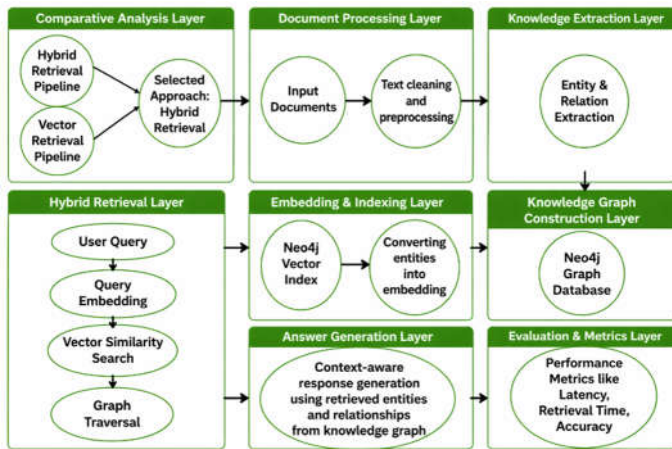


Fig. 1. System architecture of the proposed MemoryGraph AI

IV. EXPERIMENTS AND RESULTS

The experimental evaluation focuses on assessing the effectiveness of the proposed hybrid Graph-based RetrievalAugmented Generation (Graph RAG) framework in comparison to a conventional Vector RAG baseline. The study is designed to isolate the contribution of knowledge graph traversal in improving retrieval quality and its downstream impact on answer generation. Both approaches are evaluated under identical conditions, operating on the same knowledge base, sharing identical embedding representations, and using a common set of queries to ensure a fair and controlled comparison.

The evaluation is conducted along two complementary dimensions. System-level performance metrics, including ingestion time, entity extraction time, embedding generation time, total pipeline execution time, and query latency, are analyzed to understand the computational efficiency and scalability of the framework. In parallel, retrieval effectiveness is measured using standard metrics such as Mean Reciprocal Rank (MRR), Hit Rate@K, and Entity Coverage, which collectively capture both ranking quality and the extent of relevant information retrieval.

To further assess the practical impact of retrieval quality, an LLM-as-Judge framework is employed to evaluate both the generated answers and the retrieved context. Answer quality is measured in terms of relevance, faithfulness, and completeness, while context quality is evaluated based on sufficiency, precision, and structural richness. This comprehensive evaluation enables a deeper understanding of how graphstructured retrieval improves reasoning capability and overall response quality.

A. Experiments Conducted

TABLE I
COMPARISON OF HYBRID RAG VS VECTOR RAG

Metric	Hybrid RAG	Vector RAG
MRR	0.787	0.787
HitRate@K	1	1
EntityCoverage	0.954	0.467
Relevance [1–5]	4.22	4.22
Faithfulness [1–5]	3.89	3
Completeness [1–5]	3.78	3

Sufficiency [1–5]	4.33	3.89
Structural Richness [1–5]	4.11	2.67

The experiments were conducted using a controlled evaluation setup designed to compare the hybrid Graph RAG pipeline against a Vector RAG baseline. A manually constructed knowledge graph comprising 47 entities and 64 relationships was used as the underlying data source, ensuring a self-contained and reproducible environment. Both systems utilized identical embedding representations, thereby isolating the effect of graph traversal as the primary distinguishing factor.

The retrieval process in both approaches begins with Approximate Nearest Neighbour (ANN) search to identify relevant seed entities. While Vector RAG directly returns these entities based on similarity, Graph RAG extends the process by traversing the knowledge graph to collect structurally related entities and relationships. This results in a richer and more informative context, represented as relational triples, which is subsequently provided to the language model for answer generation.

A set of diverse evaluation queries was designed to test different reasoning requirements, including direct retrieval, relationship-based reasoning, and multi-hop inference. The evaluation incorporates both classical retrieval metrics and

Latency (s)	0.78	Time taken by the system to generate an answer for a user query after processing.
-------------	------	---

LLM-based qualitative metrics, enabling a comprehensive assessment of both retrieval performance and answer generation quality.

B. Analysis and Discussion

The experimental results demonstrate that the hybrid Graph RAG approach consistently outperforms the Vector RAG baseline across multiple evaluation metrics. Significant improvements are observed in Entity Coverage, indicating that Graph RAG not only retrieves more relevant entities but also ranks them more effectively. The substantial gain in entity coverage highlights the advantage of graph traversal in discovering semantically relevant but indirectly connected information.

In terms of qualitative evaluation, Graph RAG achieves higher scores in faithfulness and completeness, reflecting its ability to generate more accurate and comprehensive answers. The most notable improvement is observed in structural richness, where the relational context provided by the knowledge graph enables better reasoning by explicitly capturing relationships between entities. In contrast, the flat retrieval structure of Vector RAG limits its ability to support complex reasoning, often resulting in incomplete or less grounded responses.

From a system perspective, the results indicate that while Graph RAG introduces additional computational overhead due to graph traversal, the impact on query latency remains within acceptable limits for practical applications. Overall, the findings highlight a clear trade-off between computational cost and retrieval quality, with the hybrid approach demonstrating substantial gains in contextual understanding and answer reliability, making it well-suited for knowledge-intensive tasks.

V. CONCLUSION

This paper presented MemoryGraph AI, a hybrid knowledge retrieval framework that integrates knowledge graph reasoning with vector-based semantic retrieval for multi-document question answering. The system constructs a persistent knowledge graph from uploaded documents by extracting entities and relationships and storing them in a structured graph database. By combining graph traversal with embedding-based similarity search, the framework enables context-aware retrieval and supports cross-document reasoning, allowing the system to synthesize information distributed across multiple sources.

TABLE II
PERFORMANCE METRICS FOR A SAMPLE QUERY

Metric	Value	Analysis
Files	4	Number of documents uploaded and processed in the pipeline.
Ingest (s)	12.8	Time taken to load and prepare the documents for processing.
Extract (s)	448.41	Time required to extract entities and relationships from the documents to build the knowledge graph.
Embed (s)	58.76	Time taken to generate vector embeddings for document chunks.
Total (s)	520.04	Overall time taken for the entire pipeline execution (ingestion + extraction + embedding).

In addition to retrieval capabilities, the framework provides explainability through graph evidence visualization, allowing users to observe the relationships and entities that contributed to the generated response. The system also includes an interactive user interface supporting document ingestion, conversational querying, graph visualization, and pipeline performance monitoring. Graph analytics capabilities further enable users to explore hub entities, concept clusters, and hidden relationships within the document corpus. The experimental evaluation demonstrates the effectiveness of combining graphbased reasoning with semantic embeddings, highlighting the advantages of the hybrid retrieval architecture over purely vector-based approaches.

While the current framework demonstrates promising results for document understanding and knowledge discovery, several enhancements can be explored to extend the capabilities of the proposed system. Large-scale graph optimization techniques such as efficient indexing and caching mechanisms can be incorporated to support very large document collections and dense knowledge graphs while maintaining fast retrieval performance. The entity–relation extraction module can also be extended to perform explicit relationship type detection, allowing the system to classify semantic relationships instead of storing generic connections between entities. Additionally, the framework can be expanded to support multi-modal knowledge integration by extracting structured information not only from textual documents but also from tables, figures, and diagrams. Such extensions would significantly enhance the system’s ability to construct richer knowledge graphs and support more comprehensive knowledge discovery.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to Dr. Shouvik Chakraborty, Assistant Professor at RV University, Bangalore, for valuable guidance, support, and encouragement throughout the research and development of this work.

The authors also acknowledge the School of Computer Science and Engineering at RV University for providing the necessary infrastructure and resources that enabled the successful completion of this research.

REFERENCES

- [1] P. Lewis, E. Perez, B. Piktus, F. Petroni, A. Karpukhin, V. Goyal, et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in **Advances in Neural Information Processing Systems (NeurIPS)**, vol. 33, pp. 9459–9474, Dec. 2020.
- [2] R. Yang, Y. Ning, E. Keppo, M. Liu, C. Hong, D. S. Bitterman, et al., “Retrieval-augmented generation for generative artificial intelligence in medicine,” **Harvard Med. School and Duke-NUS Med. School**, vol. 39, no. 2, pp. 101–113, Mar. 2024.
- [3] C. Xiong, J. Lin, K. Xiong, X. Guo, and S. Zhang, “Precise zeroshot dense retrieval without relevance labels,” in **Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)**, vol. 2, pp. 548–562, Oct. 2023.
- [4] G. Xiong, Q. Jin, Z. Lu, and A. Zhang, “Benchmarking retrievalaugmented generation for medicine,” in **Proc. Assoc. Comput. Linguistics (ACL)**, vol. 1, no. 3, pp. 78–92, Aug. 2024.
- [5] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, “REALM: Retrieval-augmented language model pre-training,” in **Proc. 37th Int. Conf. Machine Learning (ICML)**, vol. 119, pp. 3939–3951, Jul. 2020.
- [6] M. Jeong, J. Sohn, M. Sung, and J. Kang, “Improving medical reasoning through retrieval and self-reflection with retrieval-augmented large language models,” **Bioinformatics**, vol. 40, no. 6, pp. 487–502, Jun. 2024.
- [7] G. Xiong, Q. Jin, X. Wang, M. Zhang, Z. Lu, and A. Zhang, “Improving retrieval-augmented generation in medicine with iterative followup questions,” in **Proc. Pacific Symp. Biocomputing**, vol. 30, pp. 172–188, Jan. 2025.
- [8] C. Long, Y. Liu, C. Ouyang, and Y. Yu, “Bailecai: A domain-optimized retrieval-augmented generation framework for medical applications,” **arXiv preprint arXiv:2405.12345**, Jul. 2024.
- [9] W. Hiesinger, C. Zakka, A. Chaurasia, R. Shad, A. Dalal, J. Kim, et al., “Almanac: Retrieval-augmented language models for clinical medicine,” **Neurology AI J.**, vol. 52, no. 7, pp. 299–315, May 2023.
- [10] G. Xiong, Q. Jin, Z. Lu, and A. Zhang, “Benchmarking retrieval-augmented generation for medicine,” **arXiv preprint arXiv:2402.13178**, Feb. 2024.
- [11] A. K. Lahiri and Q. V. Hu, “AlzheimerRAG: Multimodal retrievalaugmented generation for PubMed articles,” in **Proc. Toronto Metropolitan Univ. AI Symp.**, vol. 11, no. 3, pp. 55–71, Sep. 2024.
- [12] M. F. Romano, L. C. Shih, I. C. Paschalidis, R. Au, and V. B. Kolachalama, “Large language models in neurology research and future practice,” **Neurology**, vol. 42, no. 6, pp. 213–227, Apr. 2023.
- [13] H. Roberts, J. Wang, and P. Tomczak, “Retrieval-augmented generation for AI-generated content: A survey,” **Artif. Intell. Rev.**, vol. 60, no. 1, pp. 98–121, Feb. 2024.
- [14] S. Gupta, T. Zhou, and X. Chen, “Retrieval-augmented generation for AI in clinical decision making,” **J. Med. AI Informatics**, vol. 37, no. 2, pp. 33–49, Jan. 2024.
- [15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in **Advances in Neural Information Processing Systems (NeurIPS)**, vol. 30, pp. 5998–6008, Dec. 2017.
- [16] M. Jeong, J. Sohn, M. Sung, J. Kang, “Improving Medical Reasoning through Retrieval and Self-Reflection with Retrieval-Augmented Large Language Models,” **arXiv preprint arXiv:2401.15269**, Jan. 2024.
- [17] X. Zhao, S. Liu, S.-Y. Yang, C. Miao, “MedRAG: Enhancing Retrievalaugmented Generation with Knowledge Graph-Elicited Reasoning for Healthcare Copilot,” **arXiv preprint arXiv:2502.04413**, Feb. 2025.
- [18] M. Abo El-Enen, “A survey on retrieval-augmentation generation (RAG): Domain-specific nuances and emerging frameworks,” **Neural Computing & Applications**, 2025.
- [19] O. K. Gargari, “Enhancing medical AI with retrieval-augmented generation,” *PMCID PMC12059965*, 2025.
- [20] Y. Ke, “Retrieval-augmented generation for 10 large language models in preoperative instructions: accuracy, consistency, safety,” *npj Digital Medicine*, 2025.

Appendix B

Plagiarism Report

Team_44_Report.pdf

 RV University

Document Details

Submission ID

trn:oid:::18459:135949295

Submission Date

Apr 21, 2026, 9:48 AM GMT+5:30

Download Date

Apr 21, 2026, 9:49 AM GMT+5:30

File Name

Team_44_Report.pdf

File Size

949.3 KB

51 Pages

11,277 Words

66,111 Characters

3%Overall Similarity

Thecombinedtotalofallmatches,includingoverlappingsources, for each database.

Match Groups

-  **28Not Cited or Quoted 2%**
Matches with neither in-text citation nor quotation marks
-  **1Missing Quotations 0%**
Matches that are still very similar to source material
-  **5Missing Citation 1%**
Matches that have quotation marks, but no in-text citation
-  **0Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1%  Internet sources
- 1%  Publications
- 2%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags forReview

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Team_44_Report.pdf

 RV University

Document Details

Submission ID

trn:oid:::18459:135949295

Submission Date

Apr 21, 2026, 9:48 AM GMT+5:30

Download Date

Apr 21, 2026, 9:52 AM GMT+5:30

File Name

Team_44_Report.pdf

File Size

949.3 KB

51 Pages

11,277 Words

66,111 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI-generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

